COMP2711 / DISCRETE MATHEMATICAL TOOLS

Algorithms

Past Paper Solutions

68	1	2013-25
verified exercises	topic groups	source papers

English solutions aligned with the supplied official answers.

Each answer has two parts: the original Official Answer and the existing Supplementary Explanation. Source PDF pages are retained for verification.

Generated 2 October 2026

All verified, in-scope questions linked to Algorithms in the knowledge base.

Each question is included once, including cross-topic applications.

READING GUIDE

Contents

Algorithms
68 exercises
A001-A068

How to use this booklet

Questions are grouped by their first Algorithms subtopic, then ordered by difficulty and year (newest first). Each question appears once as A001-A068. Difficulty ranges from 1 (introductory) to 4 (challenging).

Official Answer reproduces the answer in the source paper, with its physical PDF page reference.

Supplementary Explanation retains the existing independently authored reasoning, including the stated result and derivation. The two parts remain separate throughout the booklet.

Source references use physical PDF pages. File links work with the local source collection at its existing location.

The answer provenance shown for each exercise retains its knowledge-base classification; attaching an official quotation does not reclassify a derived correction as official.

A001

Algorithms

2022 Midterm | Question 10(a) | Source page 4 | Difficulty 1
Q-2022-MID-P10-a-best | Answer provenance: official

> Determine the best-case and worst-case running time of the following algorithms using the Θ notation. No justification is needed.
>
> The following algorithm finds the maximum of a sequence $\{a_1, a_2, \ldots, a_n\}$ of n integers:
>
> ```
> max ← a_1
> for i ← 2 to n
>     if max < a_i then max ← a_i
> return max
> ```
>
> Determine the best-case running time using the Θ notation.

Official Answer

Source: physical PDF page 5 (2022-midterm.pdf).

> (a) The number of comparisons does not depend on the values of a_1 through a_n. Exactly $n - 1$ comparisons are used. Therefore, the best and worst case performance are both $\Theta(n)$.

Supplementary Explanation

Answer. $\Theta(n)$.

Derivation/proof. If the first element is already maximal, no update is executed, but the for-loop still compares max with each of the remaining n−1 elements. Each comparison and loop step takes constant time, so even the fastest input costs Θ(n). The algorithm contains no early return based on discovering the maximum.

There is no early-return branch; exactly $n - 1$ comparisons occur.

A002

Algorithms

2022 Midterm | Question 10(a) | Source page 4 | Difficulty 1
Q-2022-MID-P10-a-worst | Answer provenance: official

> Determine the best-case and worst-case running time of the following algorithms using the Θ notation. No justification is needed.
>
> The following algorithm finds the maximum of a sequence $\{a_1, a_2, \ldots, a_n\}$ of n integers:
>
> ```
> max ← a_1
> for i ← 2 to n
>     if max < a_i then max ← a_i
> return max
> ```
>
> Determine the worst-case running time using the Θ notation.

Official Answer

Source: physical PDF page 5 (2022-midterm.pdf).

> (a) The number of comparisons does not depend on the values of a_1 through a_n. Exactly $n - 1$ comparisons are used. Therefore, the best and worst case performance are both $\Theta(n)$.

Supplementary Explanation

Answer. $\Theta(n)$.

Derivation/proof. For a strictly increasing input, every one of the n−1 comparisons succeeds and performs an assignment. This adds at most constant work per iteration. There are exactly n−1 iterations on every input, so total cost is both O(n) and Ω(n), yielding Θ(n) even when every update occurs.

A strictly increasing sequence attains the most updates. The upper bound is $O(n)$ and the mandatory comparisons give $\Omega(n)$.

A003

Algorithms

2022 Midterm | Question 10(b) | Source page 4 | Difficulty 1
Q-2022-MID-P10-b-best | Answer provenance: official

> The following algorithm finds an element x in a list of n integers:
>
> ```
> for i ← 1 to n
>     if x = a_i then return i
> return “not found”.
> ```
>
> Determine the best-case and worst-case running time using the Θ notation. No justification is needed.
>
> Determine the best-case running time using the Θ notation.

Official Answer

Source: physical PDF page 5 (2022-midterm.pdf).

> (b) In the best case $x = a_1$. Thus, the best-case running time is $\Theta(1)$. In the worst case $x = a_n$. The worst-case running time is $\Theta(n)$.

Supplementary Explanation

Answer. $\Theta(1)$.

Derivation/proof. Choose an input with x=a1. The first comparison succeeds and the return exits the entire procedure, so no later list element is inspected. A constant number of initialization, comparison and return operations suffice. At least one comparison is required in the displayed nonempty-input procedure, giving a matching constant lower bound.

Choose $x = a_1$; no later elements are read.

A004

Algorithms

2022 Midterm | Question 10(b) | Source page 4 | Difficulty 1
Q-2022-MID-P10-b-worst | Answer provenance: official

> The following algorithm finds an element x in a list of n integers:
>
> ```
> for i ← 1 to n
>     if x = a_i then return i
> return “not found”.
> ```
>
> Determine the best-case and worst-case running time using the Θ notation. No justification is needed.
>
> Determine the worst-case running time using the Θ notation.

Official Answer

Source: physical PDF page 5 (2022-midterm.pdf).

> (b) In the best case $x = a_1$. Thus, the best-case running time is $\Theta(1)$. In the worst case $x = a_n$. The worst-case running time is $\Theta(n)$.

Supplementary Explanation

Answer. $\Theta(n)$.

Derivation/proof. If x is absent, every one of the n comparisons fails before the not-found return. If x appears only at an, the procedure likewise performs n tests. These exhibit Ω(n) work, while the loop has at most n iterations each of constant cost, giving O(n). Together they establish Θ(n).

For an absent target, all n comparisons fail, giving $\Omega(n)$; the total is $\Theta(n)$.

A005

Algorithms

2022 Midterm | Question 10(c) | Source page 4 | Difficulty 1
Q-2022-MID-P10-c-best | Answer provenance: official

> The following algorithm finds x in an sorted list of n integers (note that instead of comparing with the median, this algorithm compares with the 25%-quantile in each iteration):
>
> ```
> i ← 1
> j ← n
> while i ≤ j
>     m ← floor((3/4)i + (1/4)j)
>     if x = a_m then return m
>     if x > a_m then i ← m + 1
>     else j ← m − 1
> return “not found”
> ```
>
> Determine the best-case and worst-case running time using the Θ notation. No justification is needed.
>
> Determine the best-case running time using the Θ notation.

Official Answer

Source: physical PDF page 5 (2022-midterm.pdf).

> (c) The best-case running time is $\Theta(1)$. Since each iteration reduces $j - i$ by $1/4$, the worst-case running time is $\Theta(\log_{3/4} \frac{1}{n}) = \Theta(\log n)$.

Supplementary Explanation

Answer. $\Theta(1)$.

Derivation/proof. The first probe is m=floor(3/4+n/4) with i=1,j=n. Choose x=a_m. The equality branch returns at that first iteration, after only constant initialization and arithmetic. The position being a quartile rather than the median does not prevent an immediate successful match, so the best case is constant.

The first probe is $m = \lfloor 3/4 + n/4 \rfloor$; choose $x = a_m$.

A006

Algorithms

2021 Midterm | Question 8(d) | Source page 3 | Difficulty 1 | 3 marks
Q-2021-MID-P08-d | Answer provenance: official

> Consider the following three options.
>
> (i) $f(x) = \Theta(g(x))$. (ii) $f(x) = O(g(x))$. (iii) $f(x) = \Omega(g(x))$.
>
> For each of the following f, g pairs, choose all options that apply from above. No justification is necessary. Note that it is possible that none of them apply.
>
> $f(x) = \log_3(x), g(x) = \log_{20}(x)$.

Official Answer

Source: physical PDF page 4 (2021-midterm.pdf).

> (d) (i)(ii)(iii), $f(x) = \Theta(g(x))$. $\log_{20}(x) = \log_3(x) / \log_3 20$ where $1/\log_3 20$ is a constant.

Supplementary Explanation

Answer. (i), (ii), and (iii).

Derivation/proof. For x>1, change of base gives $f(x) = \ln x / \ln 3$ and $g(x) = \ln x / \ln 20$. Consequently f/g=ln20/ln3 is a fixed positive constant. Taking that constant (or slightly larger/smaller constants) yields both upper and lower bounds, so Θ holds and automatically includes O and Ω.

Precisely, $f(x)/g(x) = \ln 20 / \ln 3 = c > 0$, so select (i), (ii), and (iii).

A007

Algorithms

2018 Final | Question 10(b) | Source page 14 | Difficulty 1
Q-2018-FINAL-P10-b | Answer provenance: official

> Given a $n \times p$ matrix A and a $p \times m$ matrix B, the following algorithm computes the $n \times m$ product matrix $C = A \cdot B$.
>
> Matrix-Mult(A,B)
>
> ```
> for i = 1 to n do
>     for j = 1 to m do
>         C[i,j] = 0
>         for k = 1 to p do
>             C[i,j] = C[i,j] + A[i,k] · B[k,j]
>         end for
>     end for
> end for
> return C
> ```
>
> b) What is the $O(\cdot)$ asymptotic complexity of the algorithm when given two matrices both of which are of size $n \times n$?

Official Answer

Source: physical PDF page 15 (2018-final.pdf).

$O(n^3)$.

Supplementary Explanation

Answer. $O(n^3)$, tightly $\Theta(n^3)$.

Derivation/proof. There are n choices of row, n choices of column, and n inner iterations per output entry. Thus the innermost constant-cost multiply-and-add executes n^3 times. The n^2 initializations and loop overhead are no larger in order, so the total is $\Theta(n^3)$.

The three nested loops enumerate the output row, output column, and summation index. They perform $n \cdot n \cdot n = n^3$ multiplications in the square case.

A008

Algorithms

2017 Final | Question 3(b) | Source page 5 | Difficulty 1
Q-2017-FINAL-P03-b | Answer provenance: official

> For each of the following parts, find the "best" big-O notation to describe the complexity of the algorithm. Choose your answers from the following:
>
> $O(1), O(\log n), O(n), O(n \log n), O(n^2), O(n^3), \ldots, O(2^n), O(n!)$
>
> Assume that the amount of time to print any string is a constant, i.e. $O(1)$.
>
> (b) An algorithm that prints all subsets of size two of the set $\{1, 2, 3, \ldots, n\}$.

Official Answer

Source: physical PDF page 5 (2017-final.pdf).

n^2.

Supplementary Explanation

Answer. $O(n^2)$ (in fact $\Theta(n^2)$).

Derivation/proof. Each output is an unordered pair $\{i, j\}$ with $1 \le i < j \le n$. There are $\binom{n}{2} = n(n-1)/2$ such pairs. A nested enumeration over $i < j$ prints each once, with constant cost per output, so the total is $\Theta(n^2)$.

A009

Algorithms

2025 Fall Final | Question 8(i) | Source page 13 | Difficulty 2 | 1 mark
Q-2025F-FINAL-P08-i | Answer provenance: official

> We have two algorithms, A and B. Let $T_A(n)$ and $T_B(n)$ denote the time complexities of algorithms A and B respectively. For each case, decide whether A is faster, B is faster, or undetermined (i.e., the information provided is not enough to conclude which algorithm is faster). No explanation is needed.
>
> (i) [1 point] $T_A(n) = \Theta(\sum_{i=1}^{n} n^2) \quad T_B(n) = \Theta(n^4)$

Official Answer

Source: physical PDF page 13 (COMP2711_F2025_Final.pdf).

(i) A is faster

Supplementary Explanation

Answer. A is faster.

Derivation/proof. The summation index is i, but each term is n^2, so $\sum_{i=1}^{n} n^2 = n \cdot n^2 = n^3$. Consequently $T_A = \Theta(n^3)$ and $T_B = \Theta(n^4)$. Their ratio is bounded by constant multiples of $1/n \to 0$. Algorithm A has a strictly smaller asymptotic growth rate and is therefore faster.

A010

Algorithms

2025 Fall Final | Question 8(ii) | Source page 13 | Difficulty 2 | 1 mark
Q-2025F-FINAL-P08-ii | Answer provenance: official

> We have two algorithms, A and B. Let $T_A(n)$ and $T_B(n)$ denote the time complexities of algorithms A and B respectively. For each case, decide whether A is faster, B is faster, or undetermined (i.e., the information provided is not enough to conclude which algorithm is faster). No explanation is needed.
>
> (ii) [1 point] $T_A(n) = O(\sum_{i=1}^{n} n^2) \quad T_B(n) = \Omega(n^{3+\epsilon}) \quad (\epsilon > 0)$

Official Answer

Source: physical PDF page 13 (COMP2711_F2025_Final.pdf).

(ii) A is faster

Supplementary Explanation

Answer. A is faster.

Derivation/proof. First, $\sum_{i=1}^{n} n^2 = n^3$. By the definitions of O and Ω, positive constants C, c exist such that, for sufficiently large n, $T_A(n) \leq Cn^3$ and $T_B(n) \geq cn^{3+\epsilon}$. Therefore

$$\frac{T_A(n)}{T_B(n)} \leq \frac{C}{c} n^{-\epsilon} \to 0,$$

because $\epsilon > 0$. Even if A's actual running time is below its upper bound, it can only be faster. Thus these bounds suffice to establish that A is asymptotically faster.

A011

Algorithms

2025 Fall Final | Question 8(iii) | Source page 13 | Difficulty 2 | 1 mark
Q-2025F-FINAL-P08-iii | Answer provenance: official

> We have two algorithms, A and B. Let $T_A(n)$ and $T_B(n)$ denote the time complexities of algorithms A and B respectively. For each case, decide whether A is faster, B is faster, or undetermined (i.e., the information provided is not enough to conclude which algorithm is faster). No explanation is needed.
>
> (iii) [1 point] $T_A(n) = \Omega(\sqrt{n}) \quad T_B(n) = O(\log n)$

Official Answer

Source: physical PDF page 13 (COMP2711_F2025_Final.pdf).

(iii) B is faster

Supplementary Explanation

Answer. B is faster.

Derivation/proof. For positive constants c, C and sufficiently large n, $T_A(n) \geq c\sqrt{n}$ and $T_B(n) \leq C \log n$. Hence

$$\frac{T_B(n)}{T_A(n)} \leq \frac{C}{c} \frac{\log n}{\sqrt{n}} \to 0.$$

Here $(\log n)/\sqrt{n} \to 0$, so B's upper bound is already asymptotically smaller than A's lower bound. Their actual running times obey the same separation, and B is faster.

A012

Algorithms

2025 Fall Final | Question 8(v) | Source page 13 | Difficulty 2 | 1 mark
Q-2025F-FINAL-P08-v | Answer provenance: official

> We have two algorithms, A and B. Let $T_A(n)$ and $T_B(n)$ denote the time complexities of algorithms A and B respectively. For each case, decide whether A is faster, B is faster, or undetermined (i.e., the information provided is not enough to conclude which algorithm is faster). No explanation is needed.
>
> (v) [1 point] $T_A(n) = \Theta(n^3) \quad T_B(n) = \Theta(n^2 \log(n!))$

Official Answer

Source: physical PDF page 13 (COMP2711_F2025_Final.pdf).

(v) A is faster

Supplementary Explanation

Answer. A is faster.

Derivation/proof. We have $\log(n!) = \Theta(n \log n)$. The upper bound follows from $\log j \leq \log n$ for all $1 \leq j \leq n$. For a lower bound, the final half contains at least $\lfloor n/2 \rfloor$ terms, each at least $\log(n/2)$. Thus $T_B = \Theta(n^3 \log n)$, whereas $T_A = \Theta(n^3)$. Their ratio is $\Theta(1/\log n) \to 0$. The additional unbounded logarithmic factor makes B's time grow faster, so algorithm A is faster.

A013

Algorithms

2025 Spring Final | Question 5(a)(i) | Source page 6 | Difficulty 2 | 2 marks
Q-2025S-FINAL-P05-a-i | Answer provenance: official

> (a) [6 pts, 2 each] For each pair of expressions (A, B), indicate whether A is O, Ω, Θ, of B. Note that zero, one, or more of these relations may hold for a given pair. List all applicable relations. No explanation is needed.
>
> Examples: (a) for $A = n, B = 2n$, all O, Ω and Θ are correct; (b) for $A = n, B = n^2$, only O is correct.
>
> (i) $A = 3^{\log_2(n^2)}, \quad B = n^4$

Official Answer

Source: physical PDF page 6 (COMP2711_S2025_Final.pdf).

(a) i) O

Supplementary Explanation

Answer. Only O.

Derivation/proof. The power-log identity gives $3^{\log_2(n^2)} = (n^2)^{\log_2 3} = n^{2\log_2 3}$. Since $3 < 4$, we have $\log_2 3 < 2$, so $2\log_2 3 < 4$. Thus $A/B = n^{2\log_2 3 - 4} \to 0$. The ratio has an eventual upper bound but no positive constant lower bound. Hence $A = O(B)$, while neither Ω nor Θ holds. Both functions are positive, so the vanishing ratio makes A strictly lower order.

A014

Algorithms

2025 Spring Final | Question 5(b) | Source page 6 | Difficulty 2 | 4 marks
Q-2025S-FINAL-P05-b | Answer provenance: official

> (b) [4 pts] Consider an array A with n elements. We call a pair (i, j) an inversion, if $i < j$ and $A[i] > A[j]$. The following algorithm counts the number of inversions for array A. What is its time complexity? Write your answer in the tightest big-O notation with respect to n.
>
> Algorithm 1 CountInversions(n)

```
inversions = 0
for i = 1 to i = n−1 do
    for j = i+1 to j = n do
        if A[j] < A[i] then
            inversions = inversions+1
        end if
    end for
end for
Return inversions
```

Official Answer

Source: physical PDF page 6 (COMP2711_S2025_Final.pdf).

(b) We can see that the for $i = 1$, the second loops cost is $n - 1$, for $i = 2$ the second loop cost is $n - 2$ and $\cdots$. So we can conclude that the total cost is $(n-1) + (n-2) + \cdots + 0 = \frac{n \cdot (n-1)}{2}$ which is $O(n^2)$.

Supplementary Explanation

Answer. The tightest big-O is $O(n^2)$; the running time is $\Theta(n^2)$.

Derivation/proof. For fixed i, the inner loop runs from $i + 1$ to n, making $n - i$ comparisons. Their sum is $\sum_{i=1}^{n-1}(n-i) = (n-1) + (n-2) + \cdots + 1 = n(n-1)/2$. Each comparison and possible increment costs constant time, while initialization and outer-loop overhead are at most linear. The time is bounded above and below by positive multiples of n^2. All comparisons must still execute even when the array is already sorted.

A015

Algorithms

2024 Fall Final | Question 9(a)(i) | Source page 23 | Difficulty 2 | 1 mark
Q-2024F-FINAL-P09-a-i | Answer provenance: derived

> For each pair of expressions (A, B), indicate whether A is O, Ω, Θ, of B. Note that zero, one, or more of these relations may hold for a given pair. List all applicable relations. No explanation is needed. As an example for $A = n, B = 2n$, all O, Ω and Θ are correct.
>
> (i) $A = \log n \qquad B = 2n$

Official Answer

Source: physical PDF page 24 (COMP2711_F2024_Final.pdf).

> Answer: (a) (i) O, Ω and Θ

Supplementary Explanation

Answer. O only.

Derivation/proof. $A/B = \log n/(2n) \to 0$, because a logarithm of any fixed base greater than one grows slower than a linear function. Thus a fixed upper bound eventually gives $A = O(B)$. The ratio has no positive eventual lower bound, so $A \neq \Omega(B)$ and is not $\Theta(B)$. The printed $B = 2n$ is linear, not exponential.

A016

Algorithms

2024 Fall Final | Question 9(a)(ii) | Source page 23 | Difficulty 2 | 1 mark
Q-2024F-FINAL-P09-a-ii | Answer provenance: derived

> For each pair of expressions (A, B), indicate whether A is O, Ω, Θ, of B. Note that zero, one, or more of these relations may hold for a given pair. List all applicable relations. No explanation is needed. As an example for $A = n, B = 2n$, all O, Ω and Θ are correct.
>
> (ii) $A = 10n^2 \qquad B = 2n^2 + 10000n + 5n \log n$

Official Answer

Source: physical PDF page 24 (COMP2711_F2024_Final.pdf).

> (ii) $A = O(B)$

Supplementary Explanation

Answer. O, Ω, Θ.

Derivation/proof. Dividing by n^2 gives $A/B = 10/(2 + 10000/n + 5 \log n/n) \to 10/2 = 5$. The finite positive limit gives positive upper and lower bounds eventually, so $A = \Theta(B)$. Theta entails both big O and big Omega, and the question asks for all relations, so all three labels O, Ω, Θ must be listed. The coefficient 10000 affects only a finite range, not the asymptotic order.

A017

Algorithms

2024 Fall Final | Question 9(a)(iii) | Source page 23 | Difficulty 2 | 1 mark
Q-2024F-FINAL-P09-a-iii | Answer provenance: derived

> For each pair of expressions (A, B), indicate whether A is O, Ω, Θ, of B. Note that zero, one, or more of these relations may hold for a given pair. List all applicable relations. No explanation is needed. As an example for $A = n, B = 2n$, all O, Ω and Θ are correct.
>
> (iii) $A = n^2 \qquad B = 5^{\log_2 n}$

Official Answer

Source: physical PDF page 24 (COMP2711_F2024_Final.pdf).

> (iii) $A = \Omega(B)$

Supplementary Explanation

Answer. O only.

Derivation/proof. The identity $5^{\log_2 n} = n^{\log_2 5}$ rewrites B. Since $5 > 4$, $\log_2 5 > 2$, so $A/B = n^{2-\log_2 5} \to 0$. Thus A grows strictly slower than B, giving only $A = O(B)$, with neither Ω nor Θ. The entire $\log_2 n$ is the exponent of five; B is not five times a logarithm.

A018

Algorithms

2024 Fall Final | Question 9(a)(iv) | Source page 23 | Difficulty 2 | 1 mark
Q-2024F-FINAL-P09-a-iv | Answer provenance: derived

> For each pair of expressions (A, B), indicate whether A is O, Ω, Θ, of B. Note that zero, one, or more of these relations may hold for a given pair. List all applicable relations. No explanation is needed. As an example for $A = n, B = 2n$, all O, Ω and Θ are correct.
>
> (iv) $A = n^2 \log n \qquad B = n \log^6 n$

Official Answer

Source: physical PDF page 24 (COMP2711_F2024_Final.pdf).

> (iv) $A = O(B)$

Supplementary Explanation

Answer. Ω only.

Derivation/proof. Cancelling $n \log n$ gives $A/B = n/(\log n)^5$. A fixed logarithmic power grows more slowly than every positive power of n, so this ratio tends to infinity. Hence A eventually exceeds a positive constant multiple of B and $A = \Omega(B)$. No constant multiple of B can upper-bound A, so neither $O(B)$ nor $\Theta(B)$ holds.

A019

Algorithms

2024 Fall Final | Question 9(a)(v) | Source page 23 | Difficulty 2 | 1 mark
Q-2024F-FINAL-P09-a-v | Answer provenance: derived

> For each pair of expressions (A, B), indicate whether A is O, Ω, Θ, of B. Note that zero, one, or more of these relations may hold for a given pair. List all applicable relations. No explanation is needed. As an example for $A = n, B = 2n$, all O, Ω and Θ are correct.
>
> (v) $A = n^3(\log n)^5 \qquad B = n^{3.1}$

Official Answer

No official answer is provided in the supplied source.

Checked source: physical PDF page 24 (COMP2711_F2024_Final.pdf).

Supplementary Explanation

The published answer on physical PDF page 24 (COMP2711_F2024_Final.pdf) lists (i) through (iv) only. It contains no answer for (v); this solution is independently derived.

Answer. O only.

Derivation/proof. The ratio is $A/B = (\log n)^5/n^{0.1}$. Every fixed logarithmic power is dominated by every fixed positive power, even exponent 0.1, so the ratio tends to zero. Therefore $A = O(B)$; the positive lower bound required for Ω does not exist, so neither Ω nor Θ holds. These relations concern n tending to infinity and do not depend on numerical crossovers for small n.

A020

Algorithms

2024 Spring Midterm | Question 1 | Source page 3 | Difficulty 2 | 5 marks
Q-2024S-MID-P01 | Answer provenance: official

> Arrange the following functions in order of increasing asymptotic complexity. Note that you must write function $f(n)$ before function $g(n)$ if $f(n) = O(g(n))$. No explanation is needed.
>
> $$n,\ n\log n,\ 3^n,\ n!,\ n^2,\ 2^n,\ n^{3/2},\ \log n$$

Official Answer

Source: physical PDF page 3 (COMP2711_S2024_Midterm.pdf).

> Answer. $\log n, n, n\log n, n^{3/2}, n^2, 2^n, 3^n, n!$

Supplementary Explanation

Answer. $\log n, n, n\log n, n^{3/2}, n^2, 2^n, 3^n, n!$.

Derivation/proof. Since $\log n = o(n)$ and $\log n = o(n^{1/2})$, the eventual growth order is $\log n < n < n\log n < n^{3/2} < n^2$. Fixed-degree polynomials satisfy $n^2 = o(2^n)$, and $2^n/3^n = (2/3)^n \to 0$. The last half of the factors of $n!$ are each at least $n/2$, giving $n! \geq (n/2)^{n/2}$ for all sufficiently large n. Its ratio to 3^n is at least $(\sqrt{n/2}/3)^n \to \infty$. Thus the factorial comes last and all eight functions appear once.

A021

Algorithms

2023 Spring Midterm | Question 8(a) | Source page 12 | Difficulty 2 | 2 marks
Q-2023S-MID-P08-a | Answer provenance: official

> For each pair of expressions (A, B) below, indicate whether A is O, Ω, or Θ of B. Note that zero, one, or more of these relations may hold for a given pair. List all correct ones. It often happens that some students will get the directions wrong, so please write out the relation in full, i.e., $A = O(B)$, $A = \Omega(B)$, or $A = \Theta(B)$ and not just $O(B)$, $\Omega(B)$, or $\Theta(B)$, omitting the A.
>
> (a) $A = n^{2.9} + 100n^2$, $B = n^3$;

Official Answer

Source: physical PDF page 12 (COMP2711_S2023_Midterm.pdf).

> (a) $A = O(B)$

Supplementary Explanation

Answer. $A = O(B)$ only.

Derivation/proof. For positive n, $A/B = n^{-0.1} + 100n^{-1} \to 0$. Thus the ratio is eventually bounded above, proving $A = O(B)$. Since it becomes smaller than every positive constant, no eventual positive lower bound exists, so $A \neq \Omega(B)$ and hence $A \neq \Theta(B)$. The large coefficient 100 affects a finite range but not the limiting order.

The ratio becomes smaller than any such $c > 0$, so there is no Θ relation.

A022

Algorithms

2023 Spring Midterm | Question 8(b) | Source page 12 | Difficulty 2 | 2 marks
Q-2023S-MID-P08-b | Answer provenance: official

> For each pair of expressions (A, B) below, indicate whether A is O, Ω, or Θ of B. Note that zero, one, or more of these relations may hold for a given pair. List all correct ones. It often happens that some students will get the directions wrong, so please write out the relation in full, i.e., $A = O(B)$, $A = \Omega(B)$, or $A = \Theta(B)$ and not just $O(B)$, $\Omega(B)$, or $\Theta(B)$, omitting the A.
>
> (b) $A = 2^{2n}$, $B = 2^{3n}$;

Official Answer

Source: physical PDF page 12 (COMP2711_S2023_Midterm.pdf).

> (b) $A = O(B)$

Supplementary Explanation

Answer. $A = O(B)$ only.

Derivation/proof. The ratio is $A/B = 2^{2n-3n} = 2^{-n}$, which tends to zero. It is bounded above for all positive n, establishing $A = O(B)$. For any proposed constant $c > 0$, sufficiently large n has $2^{-n} < c$, so $A \geq cB$ cannot hold eventually. Therefore neither the Omega nor the Theta relation is valid.

The absence of a positive eventual lower bound rules out Ω and hence Θ.

A023

Algorithms

2023 Spring Midterm | Question 8(c) | Source page 12 | Difficulty 2 | 2 marks
Q-2023S-MID-P08-c | Answer provenance: official

> For each pair of expressions (A, B) below, indicate whether A is O, Ω, or Θ of B. Note that zero, one, or more of these relations may hold for a given pair. List all correct ones. It often happens that some students will get the directions wrong, so please write out the relation in full, i.e., $A = O(B)$, $A = \Omega(B)$, or $A = \Theta(B)$ and not just $O(B)$, $\Omega(B)$, or $\Theta(B)$, omitting the A.
>
> (c) $A = \log n$, $B = \log_5 n$;

Official Answer

Source: physical PDF page 12 (COMP2711_S2023_Midterm.pdf).

> (c) $A = O(B)$, $A = \Theta(B)$, and $\Omega(B)$

Supplementary Explanation

Answer. $A = O(B)$, $A = \Omega(B)$, and $A = \Theta(B)$.

Derivation/proof. Let the unsubscripted logarithm have a fixed base $b > 1$. By change of base, $\log_5 n = (\log_b n)/(\log_b 5)$. Consequently $A/B = \log_b 5$, a positive constant independent of n. This gives both a positive lower bound and a finite upper bound on the ratio; hence $A = \Theta(B)$ and both $A = O(B)$ and $A = \Omega(B)$ also hold.

Explicitly $B = \frac{\log_b n}{\log_b 5}$. The ratio is a finite positive constant, so all applicable relations must be listed, including the full expression $A = \Omega(B)$.

A024

Algorithms

2023 Spring Midterm | Question 8(e) | Source page 12 | Difficulty 2 | 2 marks
Q-2023S-MID-P08-e | Answer provenance: official

> For each pair of expressions (A, B) below, indicate whether A is O, Ω, or Θ of B. Note that zero, one, or more of these relations may hold for a given pair. List all correct ones. It often happens that some students will get the directions wrong, so please write out the relation in full, i.e., $A = O(B)$, $A = \Omega(B)$, or $A = \Theta(B)$ and not just $O(B)$, $\Omega(B)$, or $\Theta(B)$, omitting the A.
>
> (e) $A = \log\log n$, $B = 10^{100}$;

Official Answer

Source: physical PDF page 12 (COMP2711_S2023_Midterm.pdf).

> (e) $A = \Omega(B)$

Supplementary Explanation

Answer. $A = \Omega(B)$ only.

Derivation/proof. For a fixed logarithm base greater than 1 and sufficiently large n, $\log\log n$ increases without bound, whereas $B = 10^{100}$ is a fixed positive number. Therefore $A/B \to \infty$. The ratio is eventually at least 1, proving $A = \Omega(B)$, but it has no eventual finite upper bound. Hence $A \neq O(B)$ and $A \neq \Theta(B)$.

The ratio eventually exceeds any proposed finite upper-bound constant.

A025

Algorithms

2021 Midterm | Question 8(b) | Source page 3 | Difficulty 2 | 3 marks
Q-2021-MID-P08-b | Answer provenance: official

> Consider the following three options.
>
> (i) $f(x) = \Theta(g(x))$. (ii) $f(x) = O(g(x))$. (iii) $f(x) = \Omega(g(x))$.
>
> For each of the following f, g pairs, choose all options that apply from above. No justification is necessary. Note that it is possible that none of them apply.
>
> $f(x) = 2\sin(x), g(x) = |\cos(x)| + 0.5$.

Official Answer

Source: physical PDF page 4 (2021-midterm.pdf).

> (b) (ii), $f(x) = O(g(x))$. We have $f(x) \leq 4g(x)$.

Supplementary Explanation

Answer. (ii) only: $f = O(g)$.

Derivation/proof. For all real x, |f(x)|≤2 and g(x)≥1/2, so |f(x)|≤4g(x), proving O under magnitude-based bounds. At x=kπ, f(x)=0 while g(x)=3/2. These arbitrarily large zeros forbid a positive lower bound, hence Ω and Θ fail. The published selected option is also consistent with signed bounds.

The bound is $|f(x)| \leq 4g(x)$; at the zeros of f, $g(x) = 1.5$. Therefore select only (ii).

A026

Algorithms

2021 Midterm | Question 8(c) | Source page 3 | Difficulty 2 | 3 marks
Q-2021-MID-P08-c | Answer provenance: official

> Consider the following three options.
>
> (i) $f(x) = \Theta(g(x))$. (ii) $f(x) = O(g(x))$. (iii) $f(x) = \Omega(g(x))$.
>
> For each of the following f, g pairs, choose all options that apply from above. No justification is necessary. Note that it is possible that none of them apply.
>
> $f(x) = 0.5x, g(x) = x|\cos(x)|$.

Official Answer

Source: physical PDF page 4 (2021-midterm.pdf).

> (c) (iii), $f(x) = \Omega(g(x))$. $f(x) \geq 0.5g(x)$.

Supplementary Explanation

Answer. (iii) only: $f = \Omega(g)$.

Derivation/proof. For positive x, |cos x|≤1 gives g(x)≤x=2f(x), so f≥g/2 and Ω holds. At x=π/2+kπ, g(x)=0 but f(x)>0; these arbitrarily large x prevent f≤Cg for any fixed C. Thus O fails and so does Θ.

We have $g(x) \leq x = 2f(x)$, while $x = \pi/2 + k\pi$ defeats the upper bound. Therefore select only (iii).

A027

Algorithms

2021 Midterm | Question 9 | Source page 4 | Difficulty 2 | 4 marks
Q-2021-MID-P09-A | Answer provenance: official

> Given two binary numbers a and b, $a \wedge b$ is the bitwise AND operation of a and b. E.g. if $a = 3_{10} = 011_2$ and $b = 6_{10} = 110_2$, then $a \wedge b = 2_{10} = 010_2$.
>
> Consider the following two algorithms, both of which count the number of 1's in the binary representation of n.

```
procedure CountOneA(n : binary number)
    a ← 0
    while n ≠ 0
        if n mod 2 = 1 then a ← a + 1
        n ← floor(n/2)
    return a

procedure CountOneB(n : binary number)
    a ← 0
    while n ≠ 0
        a ← a + 1
        n ← n AND (n−1)
    return a
```

> For each algorithm's running time, select all that apply from the list below, and briefly justify your answer.
>
> (1) $O(\log n)$; (2) $\Omega(\log n)$; (3) $\Theta(\log n)$; (4) $O(1)$; (5) $\Omega(1)$; (6) $\Theta(1)$.
>
> For CountOneA's running time, select all that apply from the list below, and briefly justify your answer.

Official Answer

Source: physical PDF page 4 (2021-midterm.pdf).

> Algorithm A's running time is proportional to the number of bits in the binary representation of n, so it's $\Theta(\log n)$, which is also $O(\log n)$, $\Omega(\log n)$, and $\Omega(1)$.

Supplementary Explanation

Answer. (1), (2), (3), and (5).

Derivation/proof. For integer n≥1, repeated floor division by two reaches zero after exactly floor(log2 n)+1 iterations, one per binary digit. Each loop iteration uses a constant number of the source's primitive operations. Thus T(n)=Θ(log n) as n grows, implying O(log n), Ω(log n), and Ω(1). It is unbounded, so O(1) and Θ(1) do not apply.

The number of iterations is $\lfloor \log_2 n \rfloor + 1$, and the bit length is unbounded.

A028

Algorithms

2020 Final Part 1 | Question 6(a) | Source page 5 | Difficulty 2 | 2 marks
Q-2020-FINAL1-P06-a | Answer provenance: official

Read the following algorithms and answer the questions.

Algorithm 1: MultiplyTwoPolys($p(x), q(x)$) Input: A polynomial $p(x)$ with degree m and a polynomial $q(x)$ with degree n Output: The polynomial $r(x) = p(x) \cdot q(x)$

```
for i ← 0 to m do
    a_i ← the coefficient of x^i in p(x)
end
for i ← 0 to n do
    b_i ← the coefficient of x^i in q(x)
end
for i ← 0 to m+n do
    c_i ← 0
end
for i ← 0 to m do
    for j ← 0 to n do
        c_(i+j) ← c_(i+j) + a_i b_j
    end
end
r(x) ← sum from i=0 to m+n of c_i x^i
return r(x)
```

Algorithm 2: MultiplyManyPolys($p_1(x), p_2(x), \ldots, p_n(x)$) Input: Polynomials $p_1(x), p_2(x), \ldots, p_n(x)$ Output: The polynomial $r(x) = p_1(x)p_2(x)\cdots p_n(x)$

```
if n = 1 then
    return p_1(x)
end
r_1(x) ← MultiplyManyPolys(p_1(x),p_2(x),…,p_(n−1)(x))
r(x) ← MultiplyTwoPolys(r_1(x),p_n(x))
return r(x)
```

Algorithm 1 computes the multiplication of two polynomials with degree m and n respectively. What is its time complexity? Write your answer in the best big-O notation with respect to m and n.

Official Answer

Source: physical PDF page 5 (2020-final-part-1.pdf).

The time complexity of algorithm 1 is $O(mn)$.

Supplementary Explanation

Answer. $O(mn)$ as both degrees grow; more uniformly $\Theta((m+1)(n+1))$ also covers degree zero.

Derivation/proof. The coefficient reads cost Θ(m+n+2), and zeroing/output construction cost Θ(m+n+1). The nested loop performs exactly (m+1)(n+1) coefficient multiply-adds. This product dominates the linear terms, giving Θ((m+1)(n+1)) in the source's unit-cost coefficient-operation model. For m,n≥1 this is Θ(mn), whose tight big-O answer is O(mn).

For $m, n \geq 1$, the exact product count $(m+1)(n+1)$ is between mn and $4mn$. Thus the stated $\Theta(mn)$ simplification applies to positive degrees; allowing degree zero requires the unsimplified expression.

A029

Algorithms

2018 Final | Question 10(a) | Source page 14 | Difficulty 2
Q-2018-FINAL-P10-a | Answer provenance: official

> Given a $n \times p$ matrix A and a $p \times m$ matrix B, the following algorithm computes the $n \times m$ product matrix $C = A \cdot B$.
>
> Matrix-Mult(A,B)
>
> ```
> for i = 1 to n do
>     for j = 1 to m do
>         C[i,j] = 0
>         for k = 1 to p do
>             C[i,j] = C[i,j] + A[i,k] · B[k,j]
>         end for
>     end for
> end for
> return C
> ```
>
> a) Compute the output of the algorithm for input: $A = \begin{pmatrix} -1 & 1 \\ 5 & 0 \end{pmatrix}$, $B = \begin{pmatrix} 3 & 1 & 2 \\ 0 & 1 & 4 \end{pmatrix}$.

Official Answer

Source: physical PDF page 15 (2018-final.pdf).

$$C = \begin{pmatrix} -3 & 0 & 2 \\ 15 & 5 & 10 \end{pmatrix}.$$

Supplementary Explanation

Answer. $C = \begin{pmatrix} -3 & 0 & 2 \\ 15 & 5 & 10 \end{pmatrix}$.

Derivation/proof. The inner loop runs over $k = 1, 2$. Row 1 gives $C[1,1] = (-1)3 + 1 \cdot 0 = -3$, $C[1,2] = (-1)1 + 1 \cdot 1 = 0$, $C[1,3] = (-1)2 + 1 \cdot 4 = 2$. Row 2 gives $C[2,1] = 5 \cdot 3 + 0 \cdot 0 = 15$, $C[2,2] = 5 \cdot 1 + 0 \cdot 1 = 5$, $C[2,3] = 5 \cdot 2 + 0 \cdot 4 = 10$. These six loop traces form the returned array.

The result has dimensions 2×3.

For example, the first matrix entry starts with $C_{11} = (-1)3 + 1 \cdot 0 = -3$, and the final entry is $C_{23} = 5 \cdot 2 + 0 \cdot 4 = 10$. Each row-column dot product uses matching inner indices.

A030

Algorithms

2018 Final | Question 10(c) | Source page 14 | Difficulty 2
Q-2018-FINAL-P10-c | Answer provenance: official

> Given a $n \times p$ matrix A and a $p \times m$ matrix B, the following algorithm computes the $n \times m$ product matrix $C = A \cdot B$.
>
> Matrix-Mult(A,B)
>
> ```
> for i = 1 to n do
>     for j = 1 to m do
>         C[i,j] = 0
>         for k = 1 to p do
>             C[i,j] = C[i,j] + A[i,k] · B[k,j]
>         end for
>     end for
> end for
> return C
> ```
>
> c) Express the total number of multiplications that the algorithm does, expressed as a function of n, m, p.

Official Answer

Source: physical PDF page 15 (2018-final.pdf).

The first loop is executed n times, therefore the second is executed nm times, and the third nmp times. At every iteration of the innermost loop a multiplication is executed, therefore the answer is nmp.

Supplementary Explanation

Answer. nmp.

Derivation/proof. For each of n rows and m columns, the algorithm runs the inner loop p times. Every inner iteration contains exactly one scalar multiplication $A[i,k]B[k,j]$. Hence the exact multiplication count is $n \cdot m \cdot p$. Initializing to zero and adding do not contribute to the requested multiplication count.

For each output entry, sum the terms $A[i,k] \cdot B[k,j]$ over the common inner dimension. The output row is indexed by i and its column by j.

A031

Algorithms

2017 Final | Question 2 | Source page 4 | Difficulty 2 | 4 marks
Q-2017-FINAL-P02 | Answer provenance: official

> Arrange the following functions in a list so that each is big-O of the next one in the list: $n^3 + 88n^2 + 3, \log n^4, 3^n, n^2 \log n, n \cdot 2^n, 10000$.

Official Answer

Source: physical PDF page 4 (2017-final.pdf).

$10000, \log n^4, n^2 \log n, n^3 + 88n^2 + 3, n \cdot 2^n, 3^n$.

Supplementary Explanation

Answer. $10000, \log n^4, n^2 \log n, n^3 + 88n^2 + 3, n2^n, 3^n$.

Derivation/proof. Here $\log n^4 = 4 \log n$ and the cubic polynomial is $\Theta(n^3)$. Constants precede logarithms, and $\log n / n \to 0$ shows $n^2 \log n = o(n^3)$. Every fixed-degree polynomial is dominated by 2^n. Finally $(n2^n)/3^n = n(2/3)^n \to 0$, proving the last comparison.

A032

Algorithms

2017 Final | Question 3(a) | Source page 5 | Difficulty 2
Q-2017-FINAL-P03-a | Answer provenance: official

> For each of the following parts, find the "best" big-O notation to describe the complexity of the algorithm. Choose your answers from the following:
>
> $O(1), O(\log n), O(n), O(n \log n), O(n^2), O(n^3), \ldots, O(2^n), O(n!)$
>
> Assume that the amount of time to print any string is a constant, i.e. $O(1)$.
>
> (a) An algorithm that prints all bit strings of length n that end with 1.

Official Answer

Source: physical PDF page 5 (2017-final.pdf).

2^n.

Supplementary Explanation

Answer. $O(2^n)$ (in fact $\Theta(2^n)$).

Derivation/proof. The last bit is fixed and the other $n - 1$ bits have two choices each, so 2^{n-1} strings must be printed. Under the stipulated constant cost per string, this output work is $\Theta(2^{n-1}) = \Theta(2^n)$; enumeration with constant amortized work per output attains it.

A033

Algorithms

2017 Final | Question 3(c) | Source page 5 | Difficulty 2
Q-2017-FINAL-P03-c | Answer provenance: official

> For each of the following parts, find the "best" big-O notation to describe the complexity of the algorithm. Choose your answers from the following:
>
> $O(1), O(\log n), O(n), O(n \log n), O(n^2), O(n^3), \ldots, O(2^n), O(n!)$
>
> Assume that the amount of time to print any string is a constant, i.e. $O(1)$.
>
> (c) The number of print statements in the following:
>
> while $n > 1$
>
> `print "hello";` $n := \lfloor n/4 \rfloor$

Official Answer

Source: physical PDF page 5 (2017-final.pdf).

$\log n$.

Supplementary Explanation

Answer. $O(\log n)$ (in fact $\Theta(\log n)$).

Derivation/proof. After k iterations the value is $\lfloor n/4^k \rfloor$. The loop ends when this is at most one, which occurs after a number of iterations within a constant of $\log_4 n$. Every iteration prints once, so the count is logarithmic; changing the logarithm base only changes a constant factor.

Writing the starting value as n_0, the stopping condition $\lfloor n_0/4^k \rfloor \leq 1$ is equivalent to $n_0/4^k < 2$.

A034

Algorithms

2015 Final | Question 9 | Source page 15 | Difficulty 2 | 3 marks
Q-2015-FINAL-P09-recurrence | Answer provenance: official

Let A and B be two positive integers represented as binary sequences, each n bits long. Assume n is a power of 2. The product $A \cdot B$ can be computed recursively as follows:

1. If $n = 1$, calculate $A \cdot B$ directly.
2. If $n > 1$, partition the bit representation of each number into two parts, the high $n/2$ bits and low $n/2$ bits. Calculate $A \cdot B$ using the following formula:

$$\begin{aligned} A \cdot B &= (A_h \cdot 1\underbrace{0 \cdots 0}_{n/2} + A_l) \cdot (B_h \cdot 1\underbrace{0 \cdots 0}_{n/2} + B_l) \\ &= (A_h \cdot B_h) \cdot 1\underbrace{0 \cdots 0}_{n} + (A_h \cdot B_l) \cdot 1\underbrace{0 \cdots 0}_{n/2} \\ &\quad + (A_l \cdot B_h) \cdot 1\underbrace{0 \cdots 0}_{n/2} + (A_l \cdot B_l). \end{aligned} \tag{2}$$

where A_h and B_h denote the high $n/2$ bits of A and B, respectively, and A_l and B_l denote the low $n/2$ bits of A and B, respectively. The terms in ( ) are calculated recursively.

Suppose the calculations take 1 unit of time when $n = 1$, and suppose that calculation of the expression (2) takes cn units of time given the results of the recursive calls, where c is a constant.

Let $T(n)$ be the total time it takes to compute $A \cdot B$. Write a recurrence for $T(n)$, find a closed-form expression for it, and simplify the closed-form expression using the big Θ notation.

Write a recurrence for $T(n)$,

Official Answer

Source: PDF physical page 16 (2015-final.pdf).

According to the above expression of $A \cdot B$,

$$T(n) = 4T(\frac{n}{2}) + cn$$

Supplementary Explanation

Answer. $T(1) = 1, T(n) = 4T(n/2) + cn$ for powers of two $n > 1$.

Derivation/proof. Each of the four products in the expansion takes $T(n/2)$ time. Shifts and additions after these results are available cost cn by hypothesis. Therefore their sum is $4T(n/2) + cn$, with the specified direct-multiplication base case.

The four products are $A_hB_h, A_hB_l, A_lB_h, A_lB_l$. The base $T(1) = 1$ is direct multiplication, with no additional combine term.

A035

Algorithms

2015 Final | Question 9 | Source page 15 | Difficulty 2 | 2 marks
Q-2015-FINAL-P09-theta | Answer provenance: official

Let A and B be two positive integers represented as binary sequences, each n bits long. Assume n is a power of 2. The product $A \cdot B$ can be computed recursively as follows:

1. If $n = 1$, calculate $A \cdot B$ directly.
2. If $n > 1$, partition the bit representation of each number into two parts, the high $n/2$ bits and low $n/2$ bits. Calculate $A \cdot B$ using the following formula:

$$\begin{aligned} A \cdot B &= (A_h \cdot 1\underbrace{0\cdots0}_{n/2} + A_l) \cdot (B_h \cdot 1\underbrace{0\cdots0}_{n/2} + B_l) \\ &= (A_h \cdot B_h) \cdot 1\underbrace{0\cdots0}_{n} + (A_h \cdot B_l) \cdot 1\underbrace{0\cdots0}_{n/2} \\ &\quad + (A_l \cdot B_h) \cdot 1\underbrace{0\cdots0}_{n/2} + (A_l \cdot B_l). \end{aligned} \tag{2}$$

where A_h and B_h denote the high $n/2$ bits of A and B, respectively, and A_l and B_l denote the low $n/2$ bits of A and B, respectively. The terms in ( ) are calculated recursively.

Suppose the calculations take 1 unit of time when $n = 1$, and suppose that calculation of the expression (2) takes cn units of time given the results of the recursive calls, where c is a constant.

Let $T(n)$ be the total time it takes to compute $A \cdot B$. Write a recurrence for $T(n)$, find a closed-form expression for it, and simplify the closed-form expression using the big Θ notation.

and simplify the closed-form expression using the big Θ notation.

Official Answer

Source: PDF physical page 16 (2015-final.pdf).

According to the above expression of $A \cdot B$,

$$T(n) = 4T(\frac{n}{2}) + cn$$

Iterating the recurrence, we get:

$$\begin{aligned} T(n) &= T(2^j) \\ &= 4T(2^{j-1}) + 2^j c \\ &= 4(4T(2^{j-2}) + 2^{j-1}c) + 2^j c \\ &= 4^2 T(2^{j-2}) + 2 \cdot 2^j c + 2^j c \\ &= 4^2(4T(2^{j-3}) + 2^{j-2}c) + 2 \cdot 2^j c + 2^j c \\ &= 4^3 T(2^{j-3}) + 2^2 \cdot 2^j c + 2 \cdot 2^j c + 2^j c \\ &\ \ \vdots \\ &= 4^j T(1) + 2^{j-1} \cdot 2^j c + \ldots + 2 \cdot 2^j c + 2^j c \\ &= 4^j + 2^j c(2^j - 1) \\ &= (1+c)n^2 - cn \\ &= \Theta(n^2) \end{aligned}$$

Supplementary Explanation

Answer. $T(n) = \Theta(n^2)$.

Derivation/proof. From $T(n) = n^2 + cn(n-1)$ and fixed $c \geq 0$, for $n \geq 1$ we have $n^2 \leq T(n) \leq (1+c)n^2$. These positive constant bounds prove the tight quadratic order.

The leaf work alone gives $T(n) \geq n^2$.

A036

Algorithms

2014 Final | Question 9 | Source page 13 | Difficulty 2
Q-2014-FINAL-P09-theta | Answer provenance: official

> Consider a function $T(n)$ defined on integers n that are powers of 2. Suppose
>
> $$T(1) = 1, \qquad T(n) = 5T(n/2) + n^2, \quad \text{for } n > 1.$$
>
> Then simplify the closed-form expression using the big Θ notation.

Official Answer

Source: PDF physical page 13 (2014-final.pdf).

Let $n = 2^i$. Iterating the recurrence, we get

$$\begin{aligned}
T(n) &= T(2^i) \\
&= 5T(2^{i-1}) + 2^{2i} \\
&= 5[5T(2^{i-2}) + 2^{2(i-1)}] + 2^{2i} \\
&= 5^2 T(2^{i-2}) + \frac{5}{4}2^{2i} + 2^{2i} \\
&= 5^3 T(2^{i-3}) + (\frac{5}{4})^2 2^{2i} + \frac{5}{4}2^{2i} + 2^{2i} \\
&\ \ \vdots \\
&= 5^i T(2^{i-i}) + (\frac{5}{4})^{i-1} 2^{2i} + \ldots + \frac{5}{4}2^{2i} + 2^{2i} \\
&= 5^i + 2^{2i}\frac{(\frac{5}{4})^i - 1}{\frac{5}{4} - 1} \\
&= 5 \cdot 5^i - 4 \cdot 4^i \\
&= 5 \cdot n^{\log_2 5} - 4n^{\log_2 4} \\
&= \Theta(n^{\log_2 5}).
\end{aligned}$$

Supplementary Explanation

Answer. $T(n) = \Theta(n^{\log_2 5})$.

Derivation/proof. Since $\log_2 5 > 2$, the term $5n^{\log_2 5}$ dominates $4n^2$ in the closed form, leaving the stated tight order. Indeed, $n^{\log_2 5} \leq 5n^{\log_2 5} - 4n^2 \leq 5n^{\log_2 5}$ for $n \geq 1$.

Since $\log_2 5 > 2$, the upper estimate is $T(n) \leq 5n^{\log_2 5}$ and the lower estimate is $T(n) \geq n^{\log_2 5}$.

A037

Algorithms

2013 Final | Question 8 | Source page 11 | Difficulty 2
Q-2013-FINAL-P08-theta | Answer provenance: official

> Consider a function $T(n)$ defined on integers n that are powers of 2. Suppose
>
> $$T(1) = 1, \qquad T(n) = 3T(n/2) + n^2.$$
>
> Simplify the closed-form expression using the big Θ notation.

Official Answer

Source: PDF physical page 11 (2013-final.pdf).

Iterating the recurrence, we get:

$$\begin{aligned}
T(n) &= T(2^j) \\
&= 3T(2^{j-1}) + 2^{2j} \\
&= 3(3T(2^{j-2}) + 2^{2(j-1)}) + 2^{2j} \\
&= 3^2 T(2^{j-2}) + \frac{3}{4}2^{2j} + 2^{2j} \\
&= 3^2(3T(2^{j-3}) + 2^{2(j-2)}) + \frac{3}{4}2^{2j} + 2^{2j} \\
&= 3^3 T(2^{j-3}) + (\frac{3}{4})^2 2^{2j} + \frac{3}{4}2^{2j} + 2^{2j} \\
&\;\;\vdots \\
&= 3^j T(1) + (\frac{3}{4})^{j-1} 2^{2j} + \cdots + \frac{3}{4}2^{2j} + 2^{2j} \\
&= 3^j + 2^{2j}\frac{1-(3/4)^j}{1-3/4} \\
&= 3^j + 4\cdot 2^{2j} - 4\cdot 3^j = 4\cdot 2^{2j} - 3\cdot 3^j \\
&= 4n^2 - 3n^{\log_2 3} \\
&= \Theta(n^2).
\end{aligned}$$

Supplementary Explanation

Answer. $T(n) = \Theta(n^2)$.

Derivation/proof. From $T(n) = 4n^2 - 3n^{\log_2 3}$ and $\log_2 3 < 2$, the n^2 term asymptotically dominates $n^{\log_2 3}$; the positive leading coefficient establishes $\Theta(n^2)$. Explicitly, $n^2 \le 4n^2 - 3n^{\log_2 3} \le 4n^2$ for $n \ge 1$.

A038

Algorithms

2025 Fall Final | Question 8(iv) | Source page 13 | Difficulty 3 | 1 mark
Q-2025F-FINAL-P08-iv | Answer provenance: official

> We have two algorithms, A and B. Let $T_A(n)$ and $T_B(n)$ denote the time complexities of algorithms A and B respectively. For each case, decide whether A is faster, B is faster, or undetermined (i.e., the information provided is not enough to conclude which algorithm is faster). No explanation is needed.
>
> (iv) [1 point] $T_A(n) = \Omega(n^2) \quad T_B(n) = \Omega(n \log^2 n)$

Official Answer

Source: physical PDF page 13 (COMP2711_F2025_Final.pdf).

(iv) undetermind.

Supplementary Explanation

Answer. Undetermined.

Derivation/proof. Both constraints give only lower bounds. Choose $T_A = n^2, T_B = n^3$. This satisfies the given bounds, and A is faster because $n^2/n^3 \to 0$. Alternatively choose $T_A = n^3, T_B = n^2$. These also satisfy the bounds, including $n^2 = \Omega(n \log^2 n)$, but now B is faster. Two allowed implementations give opposite comparison results, so the lower bounds cannot determine which algorithm is faster.

A039

Algorithms

2025 Fall Final | Question 9(a) | Source page 14 | Difficulty 3 | 5 marks
Q-2025F-FINAL-P09-a | Answer provenance: official

> (a) [5 points] Solve the following recurrence relation using the expansion method:
>
> $$T(n) = 3T\left(\frac{n}{4}\right) + n^2 \qquad T(1) = 1$$
>
> You may assume n is of form 4^k for some positive integer k.

Official Answer

Source: physical PDF page 14 (COMP2711_F2025_Final.pdf), physical PDF page 15 (COMP2711_F2025_Final.pdf).

(a) We solve the recurrence

$$T(n) = 3T\left(\frac{n}{4}\right) + n^2, \qquad T(1) = 1,$$

where $n = 4^k$ for some positive integer k.

Using the iteration (unrolling) method:

$$\begin{aligned} T(n) &= 3T\left(\frac{n}{4}\right) + n^2 \\ &= 3\left[3T\left(\frac{n}{16}\right) + \left(\frac{n}{4}\right)^2\right] + n^2 = 3^2 T\left(\frac{n}{16}\right) + \frac{3\cdot n^2}{16} + n^2 \\ &= 3^3 T\left(\frac{n}{64}\right) + \frac{3^2\cdot n^2}{16^2} + \frac{3\cdot n^2}{16} + n^2. \end{aligned}$$

After k iterations (until the argument becomes 1, because $n/4^k = 4^k/4^k = 1$),

$$T(n) = 3^k T(1) + n^2 \sum_{i=0}^{k-1} 3^i \cdot \left(\frac{1}{16}\right)^i.$$

Since $T(1) = 1$ and $n = 4^k$, we have $3^k = (4^{\log_4 3})^k = n^{\log_4 3}$. The sum is a geometric series with ratio $r = 3/16$:

$$\sum_{i=0}^{k-1} 3^i \cdot \frac{1}{16^i} = \sum_{i=0}^{k-1} \left(\frac{3}{16}\right)^i = \frac{1-(3/16)^k}{1-3/16} = \frac{1-(3/16)^k}{13/16} = \frac{16}{13}\left(1 - \left(\frac{3}{16}\right)^k\right).$$

Thus

$$T(n) = 3^k + n^2 \cdot \frac{16}{13}\left(1 - \left(\frac{3}{16}\right)^k\right).$$

Substituting $3^k = n^{\log_4 3}$ and $(3/16)^k = 3^k/16^k = 3^k/n^2 = n^{\log_4 3 - 2}$ (or directly $(3/16)^k = 3^k/n^2$),

$$T(n) = n^{\log_4 3} + \frac{16}{13} n^2 \left(1 - \frac{n^{\log_4 3}}{n^2}\right) = n^{\log_4 3} + \frac{16}{13} n^2 - \frac{16}{13} n^{\log_4 3}.$$

Combining the terms with the same exponent,

$$T(n) = \left(1 - \frac{16}{13}\right) n^{\log_4 3} + \frac{16}{13} n^2 = -\frac{3}{13} n^{\log_4 3} + \frac{16}{13} n^2 = \frac{16n^2 - 3n^{\log_4 3}}{13}.$$

Because $\log_4 3 \approx 1.584 < 2$, the dominant term is $\frac{16}{13}n^2$, so $T(n) = \Theta(n^2)$. The exact closed form is

$$\boxed{T(n) = \frac{16n^2 - 3n^{\log_4 3}}{13}}.$$

Supplementary Explanation

Answer. $T(n) = \dfrac{16n^2 - 3n^{\log_4 3}}{13}$.

Derivation/proof. Successive substitutions give

$$T(n) = 3^{j} T(n/4^{j}) + n^{2} \sum_{i=0}^{j-1} (3/16)^{i}.$$

Write $n = 4^{k}$ and take $j = k$ to reach $T(1) = 1$. Then

$$T(n) = 3^{k} + n^{2} \frac{1 - (3/16)^{k}}{1 - 3/16}.$$

Since $16^{k} = n^{2}$, we have $n^{2}(3/16)^{k} = 3^{k}$. Consequently

$$T(n) = 3^{k} + \frac{16}{13}(n^{2} - 3^{k}) = \frac{16 n^{2} - 3 \cdot 3^{k}}{13}.$$

Use $3^{k} = n^{\log_4 3}$ to obtain the claimed formula. At $n = 4$ it gives 19, agreeing with $3T(1) + 16 = 19$. The actual value $\log_4 3 \approx 0.79248$ is below two, so the quadratic term dominates.

A040

Algorithms

2025 Fall Final | Question 9(b) | Source page 14 | Difficulty 3 | 5 marks
Q-2025F-FINAL-P09-b | Answer provenance: official

(b) [5 points] Prove your result in part (a) using induction.

Official Answer

Source: physical PDF page 15 (COMP2711_F2025_Final.pdf), physical PDF page 16 (COMP2711_F2025_Final.pdf).

(b) Let $c = \log_4 3$. We prove that for all $n = 4^k$ ($k \geq 0$),

$$T(n) = \frac{16n^2 - 3n^c}{13}.$$

Base case: $n = 4^0 = 1$

$$T(1) = 1, \qquad \frac{16 \cdot 1^2 - 3 \cdot 1^c}{13} = \frac{16 - 3}{13} = \frac{13}{13} = 1.$$

Holds.

Inductive hypothesis: Assume that for some $m = 4^\ell$ ($\ell \geq 0$),

$$T(m) = \frac{16m^2 - 3m^c}{13}.$$

Inductive step: Consider $n = 4^{\ell+1} = 4m$. By the recurrence,

$$T(n) = 3T\left(\frac{n}{4}\right) + n^2 = 3T(m) + n^2.$$

Substitute the inductive hypothesis and $n = 4m$ (so $n^2 = 16m^2$):

$$\begin{aligned} T(n) &= 3 \cdot \frac{16m^2 - 3m^c}{13} + 16m^2 \\ &= \frac{48m^2 - 9m^c}{13} + 16m^2 \\ &= \frac{48m^2 - 9m^c + 208m^2}{13} \\ &= \frac{256m^2 - 9m^c}{13}. \end{aligned}$$

Now note that $n^c = (4m)^c = 4^c m^c = (4^{\log_4 3})m^c = 3m^c$, so $9m^c = 3 \cdot 3m^c = 3n^c$. Also $256m^2 = 16 \cdot 16m^2 = 16n^2$. Therefore

$$T(n) = \frac{16n^2 - -3n^c}{13},$$

which is exactly the formula for n. By the principle of mathematical induction, the closed-form holds for all $n = 4^k$, $k \geq 0$.

Supplementary Explanation

Answer. For every $n = 4^k$ with $k \geq 0$, $T(n) = (16n^2 - 3n^{\log_4 3})/13$.

Derivation/proof. Use the recurrence $T(n) = 3T(n/4) + n^2$, the initial value $T(1) = 1$, and the candidate from part (a). Put $c = \log_4 3$. At $n = 1$, $(16 - 3)/13 = 1 = T(1)$. Assume $T(m) = (16m^2 - 3m^c)/13$ for $m = 4^k$. For $n = 4m$, substitution gives

$$T(4m) = 3(16m^2 - 3m^c)/13 + 16m^2 = (256m^2 - 9m^c)/13.$$

Since $16n^2 = 256m^2$ and $n^c = (4m)^c = 4^c m^c = 3m^c$, this equals $(16n^2 - 3n^c)/13$. The induction is complete. Keeping a single minus sign is essential; interpreting the source's double minus literally would already fail the base value.

A041

Algorithms

2025 Spring Final | Question 5(a)(ii) | Source page 6 | Difficulty 3 | 2 marks
Q-2025S-FINAL-P05-a-ii | Answer provenance: official

> (a) [6 pts, 2 each] For each pair of expressions (A, B), indicate whether A is O, Ω, Θ, of B. Note that zero, one, or more of these relations may hold for a given pair. List all applicable relations. No explanation is needed.
>
> Examples: (a) for $A = n, B = 2n$, all O, Ω and Θ are correct; (b) for $A = n, B = n^2$, only O is correct.
>
> (ii) $A = n \log n, \quad B = \log((n!)^2)$

Official Answer

Source: physical PDF page 6 (COMP2711_S2025_Final.pdf).

(a) ii) O, Ω, Θ

Supplementary Explanation

Answer. O, Ω, Θ all hold.

Derivation/proof. Expanding the product gives $B = \log((n!)^2) = 2\sum_{j=1}^{n} \log j \le 2n \log n = 2A$. At least $\lfloor n/2 \rfloor$ factors are at least $n/2$, so $B \ge 2\lfloor n/2 \rfloor \log(n/2)$. For $n \ge 4$, $\lfloor n/2 \rfloor \ge n/3$ and $\log(n/2) \ge (\log n)/2$, giving $B \ge (n/3) \log n$. Hence $A/3 \le B \le 2A$. These two-sided positive constant bounds establish all three relations.

A042

Algorithms

2025 Spring Final | Question 5(a)(iii) | Source page 6 | Difficulty 3 | 2 marks
Q-2025S-FINAL-P05-a-iii | Answer provenance: official

> (a) [6 pts, 2 each] For each pair of expressions (A, B), indicate whether A is O, Ω, Θ, of B. Note that zero, one, or more of these relations may hold for a given pair. List all applicable relations. No explanation is needed.
>
> Examples: (a) for $A = n, B = 2n$, all O, Ω and Θ are correct; (b) for $A = n, B = n^2$, only O is correct.
>
> (iii) $A = n^3 + n^2(\log n)^4, \quad B = 2^{\sqrt{n}}$

Official Answer

Source: physical PDF page 6 (COMP2711_S2025_Final.pdf).

(a) iii) O

Supplementary Explanation

Answer. Only O.

Derivation/proof. Since $(\log n)^4 = o(n)$, $A = \Theta(n^3)$. Put $t = \sqrt{n}$, so $n = t^2$ and $n^3 = t^6$. Then $A/B = \Theta(t^6/2^t) \to 0$, because an exponential outgrows every fixed polynomial. Thus $A = O(B)$, but A has no positive constant lower bound in terms of B, excluding Ω and therefore Θ.

A043

Algorithms

2023 Fall Final | Question 10 | Source page 16 | Difficulty 3 | 6 marks
Q-2023F-FINAL-P10 | Answer provenance: official

> Analyze the worst-case time complexity of the following algorithm for finding the first term of a sequence of integers equal to some previous term.
>
> ```
> procedure find duplicate(a1, a2, ..., an: integers)
>     location := 0 {no match found yet}
>     i := 2
>     while i ≤ n and location = 0
>         j := 1
>         while j < i and location = 0
>             if ai = aj then location := i
>             else j := j + 1
>         i := i + 1
>     return location
> {location is the subscript of the first value that repeats a
> previous value in the sequence and is 0 if there is no such
> value}
> ```

Official Answer

Source: physical PDF page 16 (COMP2711_F2023_Final.pdf).

> The worst case is that in which we do not find any term equal to some previous term. In that case, we need to go through all the terms a_2 through a_n, and for each of those, we need to go through all the terms occurring prior to that term. Thus the inner loop of our algorithm is executed once for $i = 2$ (namely for $j = 1$), twice for $i = 3$ (namely for $j = 1$ and $j = 2$), three times for $i = 4$, and so on, up to $n - 1$ times for $i = n$. Thus the number of comparisons that need to be made in the inner loop is $1 + 2 + 3 + \cdots + (n - 1)$. This sum is $(n - 1)(n - 1 + 1)/2$, which is clearly $O(n^2)$ but no better. Bookkeeping details do not increase this estimate.

Supplementary Explanation

Answer. Worst-case time is $\Theta(n^2)$, with exactly $n(n-1)/2$ equality comparisons on an all-distinct input.

Derivation/proof. Choose an input with all n integers distinct. Then location stays zero, the outer loop visits every $i = 2, \ldots, n$, and the inner loop compares a_i with all $i - 1$ earlier elements. The equality-comparison count is $\sum_{i=2}^{n}(i-1) = n(n-1)/2$. For any input, early discovery of a duplicate can only stop comparisons sooner, so this is also an upper bound. Each comparison and update takes constant time, with at most linear extra outer-loop bookkeeping, giving both upper and lower quadratic bounds. The official answer's O(n²) “but no better” is expressed here as the tight $\Theta(n^2)$.

The exact comparison count is $\sum_{i=2}^{n}(i-1) = n(n-1)/2$. All-distinct inputs give an attainable quadratic lower bound, and the same expression bounds the comparisons above for every input. Thus the worst-case running time is $\Theta(n^2)$. For $n = 0$ or $n = 1$, no equality comparison occurs.

A044

Algorithms

2022 Midterm | Question 9 | Source page 4 | Difficulty 3 | 6 marks
Q-2022-MID-P09 | Answer provenance: official

> Arrange the following running times in order of increasing asymptotic complexity. Just give the answer; no explanation is needed.
>
> $2^n, \sqrt{2n}, n^2 + 20, \log(n^3), \log(2^n), n^{1/3} \log n$
>
> Note that you must write function $f(n)$ before function $g(n)$ if $f(n) = O(g(n))$.

Official Answer

Source: physical PDF page 4 (2022-midterm.pdf).

> $\log(n^3),\ n^{1/3} \log n,\ \sqrt{2n},\ \log(2^n),\ n^2 + 20,\ 2^n$

Supplementary Explanation

Answer. $\log(n^3),\ n^{1/3} \log n,\ \sqrt{2n},\ \log(2^n),\ n^2 + 20,\ 2^n$.

Derivation/proof. The orders simplify to log n, n^(1/3)log n, n^(1/2), n, n², and 2^n. For the only close comparison, n^(1/3)log n divided by √(2n) equals (log n)/(√2 n^(1/6)), which tends to zero. Other adjacent ratios also tend to zero: log versus positive power, square-root versus linear, linear versus quadratic, and polynomial versus exponential. The printed radical is √(2n), not √(2^n).

The critical ratio is $\frac{n^{1/3} \log n}{\sqrt{2n}} = \frac{\log n}{\sqrt{2}\, n^{1/6}} \to 0$. The other adjacent ratios are $3/n^{1/3}$, a constant times $n^{-1/2}$, a constant times $1/n$, and $(n^2 + 20)/2^n$, all tending to zero.

A045

Algorithms

2022 Midterm | Question 10(c) | Source page 4 | Difficulty 3
Q-2022-MID-P10-c-worst | Answer provenance: official

> The following algorithm finds x in an sorted list of n integers (note that instead of comparing with the median, this algorithm compares with the 25%-quantile in each iteration):
>
> ```
> i ← 1
> j ← n
> while i ≤ j
>     m ← floor((3/4)i + (1/4)j)
>     if x = a_m then return m
>     if x > a_m then i ← m + 1
>     else j ← m − 1
> return “not found”
> ```
>
> Determine the best-case and worst-case running time using the Θ notation. No justification is needed.
>
> Determine the worst-case running time using the Θ notation.

Official Answer

Source: physical PDF page 5 (2022-midterm.pdf).

> (c) The best-case running time is $\Theta(1)$. Since each iteration reduces $j - i$ by $1/4$, the worst-case running time is $\Theta(\log_{3/4} \frac{1}{n}) = \Theta(\log n)$.

Supplementary Explanation

Answer. $\Theta(\log n)$.

Derivation/proof. For interval length L=j−i+1, m=i+floor((L−1)/4). The larger right remainder has length L−1−floor((L−1)/4), at most3L/4, so O(log n) iterations suffice. A target larger than every entry always takes this larger side, whose length is at least3(L−1)/4. Thus reaching constant size requires Ω(log n) iterations. Constant work per probe yields Θ(log n); the logarithm base4/3 affects only a constant.

For interval length $L = j - i + 1$, the left remainder is $\lfloor (L-1)/4 \rfloor$ and the larger right remainder is $L - 1 - \lfloor (L-1)/4 \rfloor$. The latter is at most $3L/4$ and at least $3(L-1)/4$. Along right branches, $L_{t+1} + 3 \geq \frac{3}{4}(L_t + 3)$; induction gives $L_t + 3 \geq (3/4)^t (n+3)$. At termination $L_t = 0$, so $t \geq \log_{4/3}((n+3)/3) = \Omega(\log n)$. Rounding affects only constants; each step does not remove exactly one quarter.

A046

Algorithms

2021 Final Part 1 | Question 2 | Source page 1 | Difficulty 3 | 10 marks
Q-2021-FINAL1-P02 | Answer provenance: official

> There are n coins, all of equal shape and size, but only $n-1$ of them are of equal weight. You are given a balance scale which can compare the weight of any two sets of coins. Assume n is a power of 3. We use the following algorithm to find the coin of different weight.
>
> ```
> Procedure findCoin(S : a set of coins)
>     if |S| = 1:
>         return the only coin in S.
>     Partition S equally into 3 sets of coins, A, B and C.
>     if A and B are of equal weight:
>         return findCoin(C)
>     else if B and C are of equal weight:
>         return findCoin(A)
>     else:
>         return findCoin(B).
> ```
>
> Show that the number of weightings is at most $2\log_3 n$.

Official Answer

Source: physical PDF page 2 (2021-final-part-1.pdf).

> In each recursive call, it takes at most 2 comparisons and the input size are reduced to one-third of the parent. So, the number of comparisons is at most $2\log_3 n = O(\log n)$.

Supplementary Explanation

Answer. At most $2\log_3 n$ weighings.

Derivation/proof. At a nontrivial call, at most two balance comparisons determine the unique group containing the odd coin. Equal A,B means C contains it; unequal A,B and equal B,C means A; if both comparisons differ, B contains it. Only that group, of size $|S|/3$, is recursed into. For n=3^k there are exactly k nontrivial size levels before size one, whose case uses no weighing. Hence total comparisons $\le 2k = 2\log_3 n$. This also establishes why only one branch is followed.

If both comparisons differ, the odd coin is in B: an odd coin in A would leave B, C balanced, and an odd coin in C would leave A, B balanced. This reasoning does not require knowing whether the coin is heavier or lighter. The successive sizes are $3^k, 3^{k-1}, \ldots, 3, 1$.

Thus the weighing bound is $2k = 2\log_3 n$.

A047

Algorithms

2021 Midterm | Question 8(a) | Source page 3 | Difficulty 3 | 3 marks
Q-2021-MID-P08-a | Answer provenance: official

> Consider the following three options.
>
> (i) $f(x) = \Theta(g(x))$. (ii) $f(x) = O(g(x))$. (iii) $f(x) = \Omega(g(x))$.
>
> For each of the following f, g pairs, choose all options that apply from above. No justification is necessary. Note that it is possible that none of them apply.
>
> $f(x) = \sin(x), g(x) = (\cos(x))^2$.

Official Answer

Source: physical PDF page 4 (2021-midterm.pdf).

> (a) None. When x keeps increasing, $(\cos(x))^2$ periodically reaches zero. When $(\cos(x))^2$ is zero, $\sin(x)$ could be -1 or 1. So, no constant c that makes $f(x) \leq c \cdot g(x)$ or $f(x) \geq c \cdot g(x)$.

Supplementary Explanation

Answer. None.

Derivation/proof. At $x = \pi/2 + 2k\pi$, |f(x)|=1 and g(x)=0, ruling out any eventual constant-factor upper bound |f|≤Cg. At x=kπ, |f(x)|=0 and g(x)=1, ruling out any positive lower bound |f|≥cg. Both sequences grow arbitrarily large. Therefore neither O nor Ω holds, and hence neither does Θ. The same counterexamples settle either signed or absolute-value comparison here.

At $x = k\pi$, $|f(x)| = 0$ while $g(x) = 1$.

A048

Algorithms

2021 Midterm | Question 8(e) | Source page 3 | Difficulty 3 | 3 marks
Q-2021-MID-P08-e | Answer provenance: official

> Consider the following three options.
>
> (i) $f(x) = \Theta(g(x))$. (ii) $f(x) = O(g(x))$. (iii) $f(x) = \Omega(g(x))$.
>
> For each of the following f, g pairs, choose all options that apply from above. No justification is necessary. Note that it is possible that none of them apply.
>
> $f(x) = 2^{\log_3 x}, g(x) = 3^{\log_{20} x}$.

Official Answer

Source: physical PDF page 4 (2021-midterm.pdf).

> (e) (iii), $f(x) = \Omega(g(x))$. $f(x) = 2^{\log_3 x} = x^{\log_3 2}$. $g(x) = 3^{\log_{20} x} = x^{\log_{20} 3}$. The answer is then obvious.

Supplementary Explanation

Answer. (iii) only: $f = \Omega(g)$.

Derivation/proof. Use $a^{\log_b x} = x^{\log_b a}$. Then f=x^α with α=ln2/ln3≈.631, and g=x^β with β=ln3/ln20≈.367. Since α>β, f/g=x^(α−β)→∞. It is eventually bounded below by a positive constant but not above, establishing Ω and excluding O and Θ. The exponents, not the outer bases alone, determine growth.

The comparison can be made exactly: $2^2 > 3$ implies $\alpha > 1/2$, whereas $3^2 < 20$ implies $\beta < 1/2$. Thus $\alpha - \beta > 0$ without relying on rounded decimal values.

Thus $f/g = x^{\alpha-\beta} \to \infty$.

A049

Algorithms

2021 Midterm | Question 9 | Source page 4 | Difficulty 3 | 4 marks
Q-2021-MID-P09-B | Answer provenance: official

> Given two binary numbers a and b, $a \wedge b$ is the bitwise AND operation of a and b. E.g. if $a = 3_{10} = 011_2$ and $b = 6_{10} = 110_2$, then $a \wedge b = 2_{10} = 010_2$.
>
> Consider the following two algorithms, both of which count the number of 1's in the binary representation of n.

```
procedure CountOneA(n : binary number)
    a ← 0
    while n ≠ 0
        if n mod 2 = 1 then a ← a + 1
        n ← floor(n/2)
    return a

procedure CountOneB(n : binary number)
    a ← 0
    while n ≠ 0
        a ← a + 1
        n ← n AND (n−1)
    return a
```

> For each algorithm's running time, select all that apply from the list below, and briefly justify your answer.
>
> (1) $O(\log n)$; (2) $\Omega(\log n)$; (3) $\Theta(\log n)$; (4) $O(1)$; (5) $\Omega(1)$; (6) $\Theta(1)$.
>
> For CountOneB's running time, select all that apply from the list below, and briefly justify your answer.

Official Answer

Source: physical PDF page 4 (2021-midterm.pdf).

> Algorithm B's running time is proportional to the number of 1's in the binary representation of n, which fluctuates between 1 and $\log n$, depending on the actual value of n. So it's $O(\log n)$ and $\Omega(1)$.

Supplementary Explanation

Answer. (1) and (5).

Derivation/proof. The update n AND (n−1) clears exactly the lowest set bit, so iterations equal popcount(n). For positive n this lies between 1 and floor(log2 n)+1, proving Ω(1) and O(log n). At n=2^k there is only one iteration, ruling out Ω(log n). At n=2^k−1 there are k iterations, ruling out O(1). Consequently neither listed Θ bound applies to the per-value running-time function.

Write the binary representation as a higher-bit prefix, the lowest one, and trailing zeros. Subtracting one changes that one to zero and the trailing zeros to ones. AND with the original therefore clears exactly that one and preserves every higher one. This justifies the popcount iteration count.

The count is popcount(n). The values $n = 2^k - 1$ show unbounded running time, so select only (1) and (5).

A050

Algorithms

2020 Final Part 1 | Question 5(b) | Source page 3 | Difficulty 3 | 5 marks
Q-2020-FINAL1-P05-b | Answer provenance: official

> Consider the following series of functions defined recursively: $f_0(x) = x^{-1}$; for $n \geq 0$, $f_{n+1}(x) = f_n'(x)$ (recall that $(ax^b)' = abx^{b-1}$).
>
> Let $a_n = \log(-f_n(-1))$. Which one of the following functions is the tight asymptotic bound (i.e., Θ) of a_n? Justify your answer.
>
> $n,\ n \log\log n,\ n \log n,\ n \log^2 n,\ n^2$

Official Answer

Source: physical PDF page 4 (2020-final-part-1.pdf).

$a_n = \log n! = \sum_{i=1}^{n} \log i$. On the one hand, $a_n < \sum_{i=1}^{n} \log n = n \log n$, so $a_n = O(n \log n)$. On the other hand, $a_n > \sum_{i=n/2+1}^{n} \log(n/2) = \frac{n}{2} \log \frac{n}{2}$, so $a_n = \Omega(n \log n)$. Therefore we conclude $a_n = \Theta(n \log n)$, and the tightest asymptotic upper bound is $n \log n$.

Supplementary Explanation

Answer. $n \log n$; $a_n = \Theta(n \log n)$.

Derivation/proof. The formula gives $f_n(-1) = (-1)^n n!(-1)^{-(n+1)} = -n!$, hence $a_n = \log(n!)$. For n≥4, all n factors of n! are at most n, so $a_n \leq n \log n$. At least n/2 factors are at least n/2, giving $a_n \geq (n/2)\log(n/2) \geq (n/4)\log n$. These fixed positive upper and lower bounds prove the claimed Θ order without needing Stirling's formula.

The constants $1/4$ and 1 with threshold 4 work for odd as well as even n and need no Stirling formula.

Substitution first gives $f_n(-1) = -n!$, so $\log(-f_n(-1)) = \log(n!)$ is defined. The lower and upper constants can be chosen as $1/4, 1$ respectively for $n \geq 4$, proving the two-sided asymptotic estimate.

A051

Algorithms

2020 Final Part 1 | Question 6(b) | Source page 5 | Difficulty 3 | 4 marks
Q-2020-FINAL1-P06-b | Answer provenance: official

Read the following algorithms and answer the questions.

Algorithm 1: MultiplyTwoPolys($p(x), q(x)$) Input: A polynomial $p(x)$ with degree m and a polynomial $q(x)$ with degree n Output: The polynomial $r(x) = p(x) \cdot q(x)$

```
for i ← 0 to m do
    a_i ← the coefficient of x^i in p(x)
end
for i ← 0 to n do
    b_i ← the coefficient of x^i in q(x)
end
for i ← 0 to m+n do
    c_i ← 0
end
for i ← 0 to m do
    for j ← 0 to n do
        c_(i+j) ← c_(i+j) + a_i b_j
    end
end
r(x) ← sum from i=0 to m+n of c_i x^i
return r(x)
```

Algorithm 2: MultiplyManyPolys($p_1(x), p_2(x), \ldots, p_n(x)$) Input: Polynomials $p_1(x), p_2(x), \ldots, p_n(x)$ Output: The polynomial $r(x) = p_1(x)p_2(x)\cdots p_n(x)$

```
if n = 1 then
    return p_1(x)
end
r_1(x) ← MultiplyManyPolys(p_1(x),p_2(x),…,p_(n−1)(x))
r(x) ← MultiplyTwoPolys(r_1(x),p_n(x))
return r(x)
```

Let $\{p_n(x)\}$ be a sequence of polynomials, all having degree 1. What is the time complexity of computing $p_1(x)p_2(x)\cdots p_n(x)$ using Algorithm 2? Write your answer in the best big-O notation with respect to n. Justify your answer.

Official Answer

Source: physical PDF page 5 (2020-final-part-1.pdf).

Let C_n be the time complexity of computing $p_1(x)p_2(x)\cdots p_n(x)$ using Algorithm 2, then $C_1 = 0$ and $C_n = C_{n-1} + n - 1$. Therefore $C_n = \sum_{i=1}^{n}(i-1) = O(n^2)$.

Supplementary Explanation

Answer. $O(n^2)$, in fact $\Theta(n^2)$.

Derivation/proof. A product of k−1 degree-one polynomials has degree k−1. Multiplying it by the kth degree-one polynomial with Algorithm 1 uses Θ(k) coefficient operations. Algorithm 2 does this sequentially for k=2,...,n, so $T(n) = T(n-1) + \Theta(n)$ with T(1)=Θ(1). Expanding sums the arithmetic sequence, yielding Θ(n²). The recursion has only one child at each step, not a balanced multiplication tree.

The last level performs exactly $2n$ coefficient multiplications, and the total for levels two through n is twice the arithmetic sum $2 + 3 + \cdots + n = n(n+1)/2 - 1$. This quadratic count yields $T(n) = \Theta(n^2)$; the displayed recurrence is an asymptotic simplification rather than the exact loop count.

A052

Algorithms

2019 Final | Question 4(a) | Source page 6 | Difficulty 3 | 4 marks
Q-2019-FINAL-P04-a | Answer provenance: official

> (a) Arrange the functions n^n, $(\log n)^2$, $n^{1.0001}$, $(1.0001)^n$, $2^{\sqrt{\log n}}$, and $n(\log n)^{1001}$ in asymptotic increasing order, i.e., each function is big-O of the next function.

Official Answer

Source: physical PDF page 6 (2019-final.pdf).

$(\log n)^2, 2^{\sqrt{\log n}}, n(\log n)^{1001}, n^{1.0001}, (1.0001)^n$ and n^n.

Supplementary Explanation

Answer. $(\log n)^2,\ 2^{\sqrt{\log n}},\ n(\log n)^{1001},\ n^{1.0001},\ (1.0001)^n,\ n^n$.

Derivation/proof. Set $t = \log n$. A power of t is dominated by $2^{\sqrt{t}}$ since $\sqrt{t}$ dominates $\log t$, while $2^{\sqrt{\log n}} = n^{o(1)}$ is smaller than n. Also $(\log n)^{1001} = o(n^{0.0001})$, giving $n(\log n)^{1001} = o(n^{1.0001})$. Every fixed polynomial is dominated by the fixed-base exponential $(1.0001)^n$. Finally its ratio to n^n is $(1.0001/n)^n \to 0$.

Taking logarithms in the first comparison gives $2\log t = o(\sqrt{t})$ for $t = \log n$. Thus every adjacent ratio tends to zero, implying the requested big-O order.

A053

Algorithms

2018 Final | Question 10(d) | Source page 14 | Difficulty 3
Q-2018-FINAL-P10-d | Answer provenance: official

> Given a $n \times p$ matrix A and a $p \times m$ matrix B, the following algorithm computes the $n \times m$ product matrix $C = A \cdot B$.
>
> Matrix-Mult(A,B)
>
> ```
> for i = 1 to n do
>     for j = 1 to m do
>         C[i,j] = 0
>         for k = 1 to p do
>             C[i,j] = C[i,j] + A[i,k] · B[k,j]
>         end for
>     end for
> end for
> return C
> ```
>
> d) Assume you are given three matrices M_1, M_2, M_3 where M_1 is 50×60, M_2 is 60×25, and M_3 is 25×8. You want to compute the product $M_1 \cdot M_2 \cdot M_3$. In which order will you do the pairwise matrix multiplications, if you want to minimize the total number of multiplications?

Official Answer

Source: physical PDF page 15 (2018-final.pdf).

There are two ways to do the multiplication. Either $M_1 \cdot M_2$ and then multiply the result with M_3 from the right, or $M_2 \cdot M_3$ and then multiply the result with M_1 from the left. For the first case, to multiply $M_1 \cdot M_2$ we need $50 \cdot 60 \cdot 25$ multiplications and the result is of size 50×25. Multiplying this with M_3 takes $50 \cdot 25 \cdot 8$ multiplications. Total multiplications is 85000. For the second case, to multiply $M_2 \cdot M_3$ we need $60 \cdot 25 \cdot 8$ multiplications and the result is of size 60×8. Multiplying this with M_1 takes $50 \cdot 60 \cdot 8$ multiplications. Total multiplications is 36000. Clearly, the second approach is much faster.

Supplementary Explanation

Answer. Compute M_2M_3 first, then $M_1(M_2M_3)$; cost 36000.

Derivation/proof. For $(M_1M_2)M_3$, the first product costs $50 \cdot 60 \cdot 25 = 75000$ and is 50×25; the second costs $50 \cdot 25 \cdot 8 = 10000$, totaling 85000. For $M_1(M_2M_3)$, the first costs $60 \cdot 25 \cdot 8 = 12000$ and is 60×8; the second costs $50 \cdot 60 \cdot 8 = 24000$, totaling 36000. These are the two parenthesizations preserving matrix order, so the second is minimal.

For three matrices in fixed order these are the only two binary parenthesizations.

Comparing the two legal parenthesizations gives $36000 < 85000$. Thus the product order with thirty-six thousand scalar multiplications is cheaper, even though both yield the same matrix.

A054

Algorithms

2018 Final | Question 12(a) | Source page 17 | Difficulty 3
Q-2018-FINAL-P12-a | Answer provenance: official

> a) Let $f_1(x), f_2(x), g(x)$ be functions from the set of real numbers to the set of positive real numbers. If $f_1(x)$ and $f_2(x)$ are both $\Theta(g(x))$, then $f_1(x) + f_2(x)$ is also $\Theta(g(x))$.

Official Answer

Source: physical PDF page 17 (2018-final.pdf).

This is true. There exist positive constraints $C_1, C_1', C_2, C_2', k_1, k_1', k_2, k_2'$, such that $f_1(x) \geq C_1 g(x)$ for all $x > k_1$, $f_1(x) \leq C_1' g(x)$ for all $x > k_1'$, $f_2(x) \geq C_2 g(x)$ for all $x > k_2$, and $f_2(x) \leq C_2' g(x)$ for all $x > k_2'$. Adding the first and third inequality, we get that $f_1(x) + f_2(x) \geq (C_1 + C_2)g(x)$ for all $x > \max\{k_1, k_2\}$. Adding the second and fourth inequality, we get that $f_1(x) + f_2(x) \leq (C_1' + C_2')g(x)$ for all $x > \max\{k_1', k_2'\}$. Hence $f_1(x) + f_2(x)$ is $\Theta(g(x))$ with constraints $(C_1 + C_2), (C_1' + C_2'), \max\{k_1, k_2\}, \max\{k_1', k_2'\}$.

Supplementary Explanation

Answer. True.

Derivation/proof. There are positive constants a_i, b_i and thresholds N_i with $a_i g(x) \leq f_i(x) \leq b_i g(x)$ for $x \geq N_i$, for $i = 1, 2$. For $x \geq \max(N_1, N_2)$, addition yields $(a_1 + a_2)g(x) \leq f_1(x) + f_2(x) \leq (b_1 + b_2)g(x)$. Both new constants are positive and independent of x, proving the required Θ bound.

Choose $N = \max(N_1, N_2)$ so that both asymptotic inequalities hold simultaneously. Beyond that threshold their constants can be combined to prove the asserted bound.

A055

Algorithms

2018 Final | Question 12(b) | Source page 17 | Difficulty 3
Q-2018-FINAL-P12-b | Answer provenance: derived

> b) Let $f(x), g(x)$ be functions from the set of real numbers to the set of positive real numbers. If $f(x)$ is $O(g(x))$ then $2^{f(x)}$ is $O(2^{g(x)})$.

Official Answer

Source: physical PDF page 17 (2018-final.pdf).

This is false. Consider $f(x) = 2x$ and $g(x) = x$. Clearly $f(x)$ is $O(g(x))$. Then $2^{f(x)} = 2^{2x} = 4^x$, whereas $2^{g(x)} = 2^x$. So $2^{f(x)}/2^{g(x)} = 2^x$, i.e., the ratio of the two functions is not bound by any constant C, therefore $2^{f(x)}$ is not $O(2^{g(x)})$.

Supplementary Explanation

Answer. False. For example $g(x) = 2^x$ and $f(x) = 2 \cdot 2^x$.

Derivation/proof. Both functions are positive for all real x, and $f = 2g$, so $f = O(g)$. But $2^{f(x)}/2^{g(x)} = 2^{f(x)-g(x)} = 2^{2^x}$, which tends to infinity. No constant can upper-bound this ratio eventually, so the proposed implication fails. The source's $f = 2x, g = x$ shows the same growth on positive x but does not satisfy the stated codomain on all reals; the displayed example does.

The counterexample must be positive everywhere on the stated real domain. Exponentials $f(x) = 2 \cdot 2^x$ and $g(x) = 2^x$ meet this condition while keeping the ratio and the desired contradiction fixed.

A056

Algorithms

2017 Final | Question 4(b) | Source page 6 | Difficulty 3
Q-2017-FINAL-P04-b-time | Answer provenance: official

> Recall that in RSA, we need to choose two primes p and q, and the public key is (n, e) where $n = pq$. Let $T = (p-1)(q-1)$. The private key is d, where $ed \equiv 1 \pmod{T}$. The encryption of x is $x^e \bmod n$. This means that d must be an integer in $\mathbb{Z}_T$ that is relatively prime with T. We can find d using the following algorithm:
>
> 1. Randomly pick $d \in \mathbb{Z}_T$.
> 2. If d and T are relatively prime, then return d. If they are not, then pick a new d and loop from step 1.
>
> Assume that we will never pick the same d twice.
>
> What is the worst case running time of your algorithm in terms of T for a single d? Express your time in big-O notation.

Official Answer

Source: physical PDF page 6 (2017-final.pdf); physical PDF page 7 (2017-final.pdf).

It is the Euclidean algorithm to find gcd: start with $\gcd(T, d)$, and the recursive step is to change $\gcd(a, b)$ into $\gcd(b, a \bmod b)$. It was shown in class that this takes $O(\log T)$ steps for the worst case of the Fibonacci numbers.

Supplementary Explanation

Answer. $O(\log T)$ arithmetic operations under the exam's unit-cost model.

Derivation/proof. For successive positive remainders $a > b > r$, write $a = qb + r$ with $q \geq 1$ and $r < b$. Hence $a \geq b + r > 2r$: after two remainder steps the value more than halves. Starting below or at T gives at most $2\log_2 T + O(1)$ steps. Consecutive Fibonacci numbers make the quotient repeatedly one and require logarithmically many steps, so this is the worst-case order.

A057

Algorithms

2017 Final | Question 4(c) | Source page 6 | Difficulty 3 | 3 marks
Q-2017-FINAL-P04-c | Answer provenance: derived

> Recall that in RSA, we need to choose two primes p and q, and the public key is (n, e) where $n = pq$. Let $T = (p-1)(q-1)$. The private key is d, where $ed \equiv 1 \pmod{T}$. The encryption of x is $x^e \bmod n$. This means that d must be an integer in $\mathbb{Z}_T$ that is relatively prime with T. We can find d using the following algorithm:
>
> 1. Randomly pick $d \in \mathbb{Z}_T$.
> 2. If d and T are relatively prime, then return d. If they are not, then pick a new d and loop from step 1.
>
> Assume that we will never pick the same d twice.
>
> (c) Bob says: "Instead of picking a random d, I will iterate over the primes from smallest to largest: I will test $d = 2, d = 3, d = 5, d = 7, \ldots$ until I get a valid d." Prove that in the worst case, this algorithm will terminate in $O(\log n)$ number of loops.

Official Answer

Source: physical PDF page 7 (2017-final.pdf).

Suppose $P_{\log_2 n}$ consists of the first $\log_2 n$ primes. Then $\prod_{x \in P_{\log_2 n}} x > \prod_{x \in P_{\log_2 n}} 2 > n$. So using the fundamental theorem of arithmetic, there cannot be more than $\log_2 n$ primes which are factors of n. This means that Bob will have to try at most $\log_2 n + 1 = O(\log n)$ primes.

Supplementary Explanation

Answer. At most $\lfloor \log_2 T \rfloor + 1 = O(\log n)$ prime candidates are tested.

Derivation/proof. A prime candidate fails precisely when it divides T. If r distinct primes fail, their product divides T and is at least 2^r. Therefore $2^r \leq T$, giving $r \leq \log_2 T$. The next tested prime succeeds, so there are at most $r + 1$ tests. Since $T = (p-1)(q-1) < pq = n$, this is $O(\log n)$. This counts candidate tests, not prime generation time.

A058

Algorithms

2015 Final | Question 9 | Source page 15 | Difficulty 3 | 5 marks
Q-2015-FINAL-P09-closed-form | Answer provenance: official

> Let A and B be two positive integers represented as binary sequences, each n bits long. Assume n is a power of 2. The product $A \cdot B$ can be computed recursively as follows:
>
> 1. If $n = 1$, calculate $A \cdot B$ directly.
> 2. If $n > 1$, partition the bit representation of each number into two parts, the high $n/2$ bits and low $n/2$ bits. Calculate $A \cdot B$ using the following formula:
>
> $$\begin{aligned} A \cdot B &= (A_h \cdot 1\underbrace{0\cdots0}_{n/2} + A_l) \cdot (B_h \cdot 1\underbrace{0\cdots0}_{n/2} + B_l) \\ &= (A_h \cdot B_h) \cdot 1\underbrace{0\cdots0}_{n} + (A_h \cdot B_l) \cdot 1\underbrace{0\cdots0}_{n/2} \\ &\quad + (A_l \cdot B_h) \cdot 1\underbrace{0\cdots0}_{n/2} + (A_l \cdot B_l). \end{aligned} \tag{2}$$
>
> where A_h and B_h denote the high $n/2$ bits of A and B, respectively, and A_l and B_l denote the low $n/2$ bits of A and B, respectively. The terms in ( ) are calculated recursively.
>
> Suppose the calculations take 1 unit of time when $n = 1$, and suppose that calculation of the expression (2) takes cn units of time given the results of the recursive calls, where c is a constant.
>
> Let $T(n)$ be the total time it takes to compute $A \cdot B$. Write a recurrence for $T(n)$, find a closed-form expression for it, and simplify the closed-form expression using the big Θ notation.
>
> find a closed-form expression for it,

Official Answer

Source: PDF physical page 16 (2015-final.pdf).

According to the above expression of $A \cdot B$,

$$T(n) = 4T(\frac{n}{2}) + cn$$

Iterating the recurrence, we get:

$$\begin{aligned} T(n) &= T(2^j) \\ &= 4T(2^{j-1}) + 2^j c \\ &= 4(4T(2^{j-2}) + 2^{j-1}c) + 2^j c \\ &= 4^2 T(2^{j-2}) + 2 \cdot 2^j c + 2^j c \\ &= 4^2(4T(2^{j-3}) + 2^{j-2}c) + 2 \cdot 2^j c + 2^j c \\ &= 4^3 T(2^{j-3}) + 2^2 \cdot 2^j c + 2 \cdot 2^j c + 2^j c \\ &\;\;\vdots \\ &= 4^j T(1) + 2^{j-1} \cdot 2^j c + \ldots + 2 \cdot 2^j c + 2^j c \\ &= 4^j + 2^j c(2^j - 1) \\ &= (1+c)n^2 - cn \\ &= \Theta(n^2) \end{aligned}$$

Supplementary Explanation

Answer. $T(n) = (1+c)n^2 - cn$.

Derivation/proof. For $j = \log_2 n$, iteration gives $T(n) = 4^j T(1) + cn \sum_{i=0}^{j-1} 2^i = n^2 + cn(n-1) = (1+c)n^2 - cn$. Substitution into $4T(n/2) + cn$ reproduces the expression, and at $n = 1$ it equals 1.

A059

Algorithms

2014 Final | Question 8 | Source page 12 | Difficulty 3 | 10 marks
Q-2014-FINAL-P08 | Answer provenance: derived

> Let a be a real number such that $a > 1$. Consider a function $T(n)$ over integers defined as follows
>
> $$T(0) = 1, \qquad T(n) = aT(n-1) + n, \quad \text{for } n > 0.$$
>
> Prove that $T(n) = \Theta(a^n)$.
>
> You might need the fact that, for any $x \neq 1$,
>
> $$\sum_{i=1}^{n} ix^i = \frac{nx^{n+2} - (n+1)x^{n+1} + x}{(1-x)^2}.$$

Official Answer

Source: PDF physical page 12 (2014-final.pdf).

By iterating the recurrence, we get

$$\begin{aligned} T(n) &= aT(n-1) + n \\ &= a^2T(n-2) + a(n-1) + n \\ &= a^3T(n-3) + a^2(n-2) + a(n-1) + n \\ &\quad \cdots \\ &= a^n + \sum_{i=1}^{n} a^{n-i}i \end{aligned}$$

Hence, we have

$$\begin{aligned} T(n) &= a^n + \sum_{i=1}^{n} a^{n-i}i \\ &= a^n + a^n \sum_{i=1}^{n} a^{-i}i \\ &= a^n + a^n \frac{n(\frac{1}{a})^{n+2} - (n+1)(\frac{1}{a})^{n+1} + (\frac{1}{a})}{(1-a)^2} \\ &= \Theta(a^n). \end{aligned}$$

Supplementary Explanation

Answer. $T(n) = \Theta(a^n)$.

Derivation/proof. Iterating yields $T(n) = a^n + \sum_{i=1}^{n} a^{n-i}i = a^n\left(1 + \sum_{i=1}^{n} i(1/a)^i\right)$. Put $x = 1/a \in (0,1)$. The supplied finite-sum formula gives $\sum_{i=1}^{n} ix^i = [x - x^{n+1}(n+1-nx)]/(1-x)^2 \le x/(1-x)^2$, since $n + 1 - nx > 0$. All its summands are nonnegative, so

$$a^n \le T(n) \le \left(1 + \frac{a}{(a-1)^2}\right) a^n.$$

The two multipliers are positive constants independent of n, proving the tight bound.

In compact notation, the upper bound is $T(n) \le [1 + a/(a-1)^2]a^n$.

A060

Algorithms

2014 Final | Question 9 | Source page 13 | Difficulty 3
Q-2014-FINAL-P09-closed-form | Answer provenance: official

> Consider a function $T(n)$ defined on integers n that are powers of 2. Suppose
>
> $$T(1) = 1, \qquad T(n) = 5T(n/2) + n^2, \quad \text{for } n > 1.$$
>
> Iterate the recurrence or use a recursion tree to find a closed-form expression for $T(n)$.

Official Answer

Source: PDF physical page 13 (2014-final.pdf).

Let $n = 2^i$. Iterating the recurrence, we get

$$\begin{aligned}
T(n) &= T(2^i) \\
&= 5T(2^{i-1}) + 2^{2i} \\
&= 5[5T(2^{i-2}) + 2^{2(i-1)}] + 2^{2i} \\
&= 5^2 T(2^{i-2}) + \frac{5}{4}2^{2i} + 2^{2i} \\
&= 5^3 T(2^{i-3}) + (\frac{5}{4})^2 2^{2i} + \frac{5}{4}2^{2i} + 2^{2i} \\
&\;\;\vdots \\
&= 5^i T(2^{i-i}) + (\frac{5}{4})^{i-1} 2^{2i} + \ldots + \frac{5}{4}2^{2i} + 2^{2i} \\
&= 5^i + 2^{2i}\frac{(\frac{5}{4})^i - 1}{\frac{5}{4} - 1} \\
&= 5 \cdot 5^i - 4 \cdot 4^i \\
&= 5 \cdot n^{\log_2 5} - 4n^{\log_2 4} \\
&= \Theta(n^{\log_2 5}).
\end{aligned}$$

Supplementary Explanation

Answer. $T(n) = 5n^{\log_2 5} - 4n^2$.

Derivation/proof. Put $n = 2^i$. Expansion gives $T(2^i) = 5^i + 4^i \sum_{j=0}^{i-1} (5/4)^j = 5^i + 4^i[(5/4)^i - 1]/(1/4) = 5 \cdot 5^i - 4 \cdot 4^i$.

At level j, the work is $5^j (n/2^j)^2 = n^2 (5/4)^j$. There are i internal levels. Substituting $5^i = n^{\log_2 5}$ gives $5n^{\log_2 5} - 4n^2$, also equal to one at $n = 1$.

A061

Algorithms

2013 Final | Question 8 | Source page 11 | Difficulty 3
Q-2013-FINAL-P08-closed-form | Answer provenance: official

> Consider a function $T(n)$ defined on integers n that are powers of 2. Suppose
>
> $$T(1) = 1, \qquad T(n) = 3T(n/2) + n^2.$$
>
> Iterate the recurrence or use a recursion tree to find a closed-form expression for $T(n)$.

Official Answer

Source: PDF physical page 11 (2013-final.pdf).

Iterating the recurrence, we get:

$$\begin{aligned}
T(n) &= T(2^j) \\
&= 3T(2^{j-1}) + 2^{2j} \\
&= 3(3T(2^{j-2}) + 2^{2(j-1)}) + 2^{2j} \\
&= 3^2 T(2^{j-2}) + \frac{3}{4}2^{2j} + 2^{2j} \\
&= 3^2(3T(2^{j-3}) + 2^{2(j-2)}) + \frac{3}{4}2^{2j} + 2^{2j} \\
&= 3^3 T(2^{j-3}) + (\frac{3}{4})^2 2^{2j} + \frac{3}{4}2^{2j} + 2^{2j} \\
&\;\;\vdots \\
&= 3^j T(1) + (\frac{3}{4})^{j-1} 2^{2j} + \cdots + \frac{3}{4}2^{2j} + 2^{2j} \\
&= 3^j + 2^{2j}\frac{1-(3/4)^j}{1-3/4} \\
&= 3^j + 4\cdot 2^{2j} - 4\cdot 3^j = 4\cdot 2^{2j} - 3\cdot 3^j \\
&= 4n^2 - 3n^{\log_2 3} \\
&= \Theta(n^2).
\end{aligned}$$

Supplementary Explanation

Answer. $T(n) = 4n^2 - 3n^{\log_2 3}$.

Derivation/proof. Write $n = 2^j$. Expansion gives $T(2^j) = 3^j + 2^{2j}\sum_{i=0}^{j-1}(3/4)^i = 3^j + 4\cdot 2^{2j} - 4\cdot 3^j = 4n^2 - 3n^{\log_2 3}$.

At level i, the 3^i subproblems each cost $(n/2^i)^2$, giving $n^2(3/4)^i$. At $j = 0$, the sum is empty and the formula gives $T(1) = 1$.

A062

Algorithms

2013 Final | Question 9 | Source page 12 | Difficulty 3 | 11 marks
Q-2013-FINAL-P09 | Answer provenance: official

> Consider a function $T(n)$ defined on integers n that are powers of 3. Suppose
>
> $$T(1) = 1, \qquad T(n) \leq 9T\left(\frac{n}{3}\right) + 4n^2 + 100n \quad \forall n > 1.$$
>
> Use advanced induction to prove that $T(n) = O(n^2 \log n)$.

Official Answer

Source: PDF physical page 12 (2013-final.pdf); PDF physical page 13 (2013-final.pdf).

Proof 1: It suffices to show that there exist n_0 and c such that

$$T(n) \leq cn^2 \log n \qquad \forall n = 3^i, n > n_0.$$

- Base case: Pick $n_0 = 1$. Then the base case is when $n = 3^1 = 3$:

  $$T(3) \leq 9T(1) + 4\,3^2 + 100 \times 3 = 345.$$

  We want the right hand side to be no greater than $c3^2 \log 3$. To satisfy the condition, we need $c \geq \frac{115}{3 \log 3}$.
- Induction Hypothesis: Suppose

  $$T(m) \leq cm^2 \log m \qquad \forall m = 3^i, m < n.$$

- Induction Step:

  $$\begin{aligned} T(n) &\leq 9T(\frac{n}{3}) + 4n^2 + 100n \\ &\leq 9c(\frac{n}{3})^2 \log \frac{n}{3} + 4n^2 + 100n \\ &= cn^2 \log n - cn^2 \log 3 + 4n^2 + 100n \\ &= cn^2 \log n \quad \text{if } c > 104/\log 3 \end{aligned}$$

  So, the proof follows through if we choose $n_0 = 1$ and $c = \max\{\frac{115}{3 \log 3}, 104/\log 3\} = 104/\log 3$. The proof is completed.

Proof 2: It suffices to show there exist positive constants c_1, c_2 and n_0 such that

$$T(n) \leq c_1 n^2 \log_3 n - c_2 n, \ \forall n > n_0. \qquad (*)$$

Base case: Let $n_0 = 1$. Remember n is a power of 3. The smallest such integer such that $n > 1$ is 3. So, the base case is $n = 3$. For this case, we have

$$T(3) \leq 9T(1) + 4 \times 3^2 + 100 \times 3 = 345 \leq 9c_1 - 3c_2.$$

This is true when $c_1 \geq \frac{3c_2 + 345}{9} = \frac{c_2 + 115}{3}$.

Induction hypothesis: Now let $n > 3$. Assume that, for any m such that $1 \leq m < n$ and m is power of 3, the following is true:

$$T(m) \leq c_1 m^2 \log_3 m - c_2 m.$$

Induction step: Now consider the case of n:

$$\begin{aligned}
T(n) &\leq 9T(\frac{n}{3}) + 4n^2 + 100n \\
&\leq 9(c_1(\frac{n}{3})^2 \log_3 \frac{n}{3} - c_2\frac{n}{3}) + 4n^2 + 100n \\
&= c_1 n^2 \log_3 n - c_1 n^2 - 3c_2 n + 4n^2 + 100n \\
&= c_1 n^2 \log_3 n - c_2 n + (4 - c_1)n^2 + (100 - 2c_2)n \\
&\leq c_1 n^2 \log_3 n - c_2 n + (4 - c_1)n^2 + (100 - 2c_2)n^2 \\
&= c_1 n^2 \log_3 n - c_2 n + (104 - c_1 - 8c_2)n^2 \\
&\leq c_1 n^2 \log_3 n - c_2 n, \quad \text{when } (104 - c_1 - 2c_2)n^2 \leq 0
\end{aligned}$$

So, we have shown that inequality (*) is true when

$$n_0 = 1, c_2 > 0, c_1 \geq \max\{\frac{c_2 + 115}{3}, 104 - 2c_2\}.$$

Consequently, $T(n) = O(n^2 \log n)$. Q.E.D

Supplementary Explanation

Answer. $T(n) \leq cn^2 \log n$ for suitable c; thus $T(n) = O(n^2 \log n)$.

Derivation/proof. For $n = 3$, $T(3) \leq 345$, so take $c \geq 115/(3 \log 3)$. Assume $T(m) \leq cm^2 \log m$ for smaller powers. Then $T(n) \leq 9c(n/3)^2 \log(n/3) + 4n^2 + 100n = cn^2 \log n - cn^2 \log 3 + 4n^2 + 100n \leq cn^2 \log n$ when $c \geq 104/\log 3$ (using $100n \leq 100n^2/3$ for $n \geq 3$).

For powers of three with $n \geq 9$, the induction hypothesis applies to $n/3 \geq 3$. Taking $c = 104/\log 3$ gives $-cn^2 \log 3 = -104n^2$, while $4n^2 + 100n \leq 104n^2$. The estimate need not hold at $n = 1$, where $\log 1 = 0$.

The base calculation is $T(3) \leq 9 + 36 + 300 = 345$.

A063

Algorithms

2025 Spring Final | Question 5(c) | Source page 6 | Difficulty 4 | 5 marks
Q-2025S-FINAL-P05-c | Answer provenance: official

(c) [5 pts] Derive the tightest possible asymptotic upper bound for the following recurrence $T(n)$, assuming n is a power of a power of 2. Do not use the master theorem. Show all your steps to receive full credit.

$$T(2) = 1, \qquad T(n) = \sqrt{n}T(\sqrt{n}) + n \quad \text{for } n > 2.$$

Official Answer

Source: physical PDF page 7 (COMP2711_S2025_Final.pdf).

(c) By expansion:

$$\begin{aligned}
T(n) &= n^{1/2}T(n^{1/2}) + n \\
&= n^{1/2}(n^{1/4}T(n^{1/4}) + n^{1/2}) + n \\
&= n^{1/2+1/4}T(n^{1/2^2}) + 2n \\
&= n^{1/2+1/4+\cdots+1/2^k}T(n^{1/2^k}) + kn \\
&= n^{1/2+1/4+\cdots+1/2^k}(1) + kn \quad [\text{Set } n^{1/2^k} = 2, \text{ we have } k = \log\log n] \\
&= n^{\frac{1/2}{1-1/2}} + kn \\
&= n + n\log\log n \\
&= O(n \cdot \log\log n).
\end{aligned}$$

Solution: $O(n \cdot \log\log n)$

Supplementary Explanation

Answer. $T(n) = n/2 + n\log_2\log_2 n$, so the tight upper bound is $O(n\log\log n)$.

Derivation/proof. After j expansions,

$$T(n) = n^{1/2+1/4+\cdots+1/2^j}T(n^{1/2^j}) + jn.$$

Each new additive contribution is $n^{1-1/2^j}n^{1/2^j} = n$. Write $n = 2^{2^k}$; then $n^{1/2^k} = 2$ and $k = \log_2\log_2 n$. The finite exponent sum is $1 - 1/2^k$, so the leaf coefficient is $n^{1-1/2^k} = n/2$. Using $T(2) = 1$ gives $T(n) = n/2 + kn$, which is $\Theta(n\log\log n)$ as k grows. At $n = 4$, the recurrence gives $T(4) = 2 \cdot 1 + 4 = 6$, agreeing with the corrected exact formula; the printed expression would give eight.

A064

Algorithms

2023 Fall Final | Question 8(a) | Source page 12 | Difficulty 4 | 7 marks
Q-2023F-FINAL-P08-a | Answer provenance: official

> For every non-negative integer n, a_{n+1} is calculated from a_n with the following rules:
>
> If the rightmost digit of a_n is bigger than 5, then $a_{n+1} = 9 \times a_n$. Otherwise, we just "remove" the rightmost digit of a_n and the remaining number is a_{n+1}. If a_{n+1} does not contain any digits we stop.
>
> As an example, if $a_n = 124$ then $a_{n+1} = 12$ as a_n's rightmost digit is 4 and we simply remove it from 124 to form 12. Also if $a_n = 16$ then $a_{n+1} = 9 \times 16 = 144$ as a_n's rightmost digit was bigger than 5.
>
> Use strong induction on a_0 to prove that this process stops in finitely many steps for any arbitrary a_0. Please write a complete induction proof consisting of base case or cases, inductive hypothesis, inductive step and conclusion.
>
> (Hint 1: If this process stops for any $1 \leq a_0 \leq k$, can you show it will stop in finitely many steps for $a_0 = k + 1$ as well?)
>
> (Hint 2: If the rightmost digit of a number a_n is larger than 5, what can we say about the possible values for the rightmost digit of $9 \times a_n$?)

Official Answer

Source: physical PDF page 12 (COMP2711_F2023_Final.pdf).

> (a) We use strong induction on the first element a_0. Obviously this process stops in 1 step if $a_0 = 1$. Assume that this assumption is correct for all values between 1 up to k. And now we prove it will be also correct for $k + 1$. If $K + 1$ ends with a digit that is smaller or equal than 5 then we just simply remove that digit and the result number is already smaller than $K + 1$ so it will stop after finitely many steps. If $K + 1$ ends with a digit that is bigger than 5, then We get $9(k + 1)$ as the next number that definitely ends with a digit smaller than 6. So for its next step we need to remove the rightmost digit that will give us a number that is again smaller than $k + 1$. (because $9(k + 1)/10$ is smaller than $k + 1$ for sure)

Supplementary Explanation

Answer. The process terminates for every nonnegative integer starting value.

Derivation/proof. For the single-digit starts 0 through 5, deletion immediately removes the only digit and terminates. Use strong induction on a positive starting integer N; assume every smaller nonnegative starting value terminates. If N ends in a digit at most 5, deletion either stops immediately or leaves $\lfloor N/10 \rfloor < N$, which terminates by the hypothesis. If its last digit is $r \in \{6, 7, 8, 9\}$, the last digit of $9N$ is $10 - r \in \{4, 3, 2, 1\}$. The next step therefore deletes that digit and leaves $\lfloor 9N/10 \rfloor < N$. This smaller value terminates by the induction hypothesis. In both cases only one or two steps precede a terminating smaller instance, proving the result for N and hence for every allowed start.

Base cases $N = 0, 1, 2, 3, 4, 5$ terminate after deleting the only digit. For $N \geq 6$, use the induction hypothesis for all smaller nonnegative starting values. If the last digit exceeds five, the two-step value is $\lfloor 9N/10 \rfloor < N$. Thus the induction step is complete.

A065

Algorithms

2021 Final Part 1 | Question 3 | Source page 2 | Difficulty 4 | 10 marks
Q-2021-FINAL1-P03 | Answer provenance: official

> Consider a company with $n > 0$ workers. A worker is a famous-solo-worker if she or he is known by every other worker, but knows none of them. You are the boss and you want to check if there is a famous-solo-worker in your company. You are only allowed to ask the following question to a worker: "Do you know worker j?" The workers will answer your questions truthfully. Show that you can find the famous-solo-worker, or declare that such a worker doesn't exist, with at most $3(n-1)$ questions.
>
> Note that there can be at most one famous-solo-worker.

Official Answer

Source: physical PDF page 2 (2021-final-part-1.pdf), physical PDF page 3 (2021-final-part-1.pdf).

> We prove this by induction.
>
> **Basis step:** When $n = 1$, the only worker must be a famous-solo-worker, and no questions are needed, which equals to $3(1-1) = 0$.
>
> **Inductive step:** The inductive hypothesis is that if there are $k \geq 1$ workers, then we can determine whether there is a famous-solo-worker with at most $3(k-1)$ questions.
>
> Suppose there are $k+1$ workers. Let i and j be any two workers among them. We ask whether i knows j, this takes one question. If i knows j, then i is not a famous-solo-worker. If i doesn't know j, then j is not a famous-solo-worker. Without loss of generality, assume that i is not a famous-solo-worker. Excluding i from the $k+1$ workers, there are k workers (including j) remaining. By the inductive hypothesis, we can use $3(k-1)$ questions for our checking.
>
> If there is no famous-solo-worker among these k workers, then we know that there is no famous-solo-worker, and the number of questions used is $1 + 3(k-1) = 3k - 2 \leq 3k$.
>
> If there is a famous-solo-worker s among these k workers, then we ask whether s knows i and whether i knows s. This takes two more questions. If i knows s and s doesn't know i, then s is still a famous-solo-worker, otherwise s is not a famous-solo-worker (thus no famous-solo-worker). In this case, we used a total of $1 + 3(k-1) + 2 = 3k$ questions.

Supplementary Explanation

Answer. Eliminate candidates with $n-1$ questions, then verify the survivor using at most $2(n-1)$ more.

Derivation/proof. Start with any candidate c. For each other worker j ask whether c knows j. If yes, c is disqualified and replace c by j; if no, j is disqualified because c does not know j. An induction over comparisons shows any genuine famous-solo-worker remains among the candidates, so n−1 questions leave only c possible. Ask c whether they know each other worker and ask each other worker whether they know c, using at most 2(n−1) questions. Accept c iff all first answers are no and all second answers yes; otherwise nobody qualifies, since all others were already disqualified. For n=1 both conditions are vacuous and zero questions suffice.

The iterative proof below uses at most $(n-1) + 2(n-1) = 3(n-1)$ questions. Each eliminated person has a concrete disqualifying answer, so failure of final verification establishes nonexistence rather than merely failure of the candidate.

Elimination never eliminates a genuine famous-solo-worker. Accept the final candidate exactly when all first-direction answers are no and all reverse-direction answers are yes.

A066

Algorithms

2019 Final | Question Bonus Problem(a) | Source page 17 | Difficulty 4 | 3 marks
Q-2019-FINAL-BONUS-a | Answer provenance: official

> Consider the following recurrence: $f_1 = 1$, $f_n = 2f_{n-1} + n$ for $n \geq 2$. The following is a purported induction proof that $f_n = O(n)$.
>
> Proof. Base case: When $n = 1$, $f(n) = O(n)$ because $f(1) = O(1)$.
>
> Induction step: Assuming $f_{n-1} = O(n-1)$, we will prove $f_n = O(n)$:
>
> $$\begin{aligned} f_n &= 2f_{n-1} + n \\ &= 2 \cdot O(n-1) + n && \text{(induction hypothesis)} \\ &= O(3n-2) \\ &= O(n) && \text{(dropping lower order term and constant coefficient)} \end{aligned}$$
>
> as desired.
>
> (a) Identify the mistake in the proof.

Official Answer

Source: physical PDF page 17 (2019-final.pdf).

The hidden constant in the induction conclusion $f_n = O(n)$ is different (larger) from the one in the induction hypothesis $f_{n-1} = O(n-1)$. Generally, one cannot use asymptotic notation in induction proofs, because the induction step is implicitly applied many times, so the constant coefficient will no longer be a constant.

Supplementary Explanation

Answer. The induction silently changes the hidden big-O constant instead of preserving one constant for all n.

Derivation/proof. An actual claim $f_n \leq Cn$ needs one fixed C and a threshold. From the induction hypothesis $f_{n-1} \leq C(n-1)$, the recurrence gives only $f_n \leq 2C(n-1) + n = (2C+1)n - 2C$. This is generally not bounded by the same Cn, since their difference is $(C+1)n - 2C$, positive for large n. Writing O again hides this growth of constants. A statement about the single value f(1) also cannot by itself establish an asymptotic bound.

A value at the single input $n = 1$ is also not by itself a valid base case for an asymptotic statement.

The same constant must work throughout the proposed induction. The unstrengthened estimate would require $(C+1)n \leq 2C$ for arbitrarily large n, which fails for each fixed C; this explains why that particular induction bound cannot close.

A067

Algorithms

2019 Final | Question Bonus Problem(b) | Source page 17 | Difficulty 4 | 7 marks
Q-2019-FINAL-BONUS-b | Answer provenance: official

> Consider the following recurrence: $f_1 = 1$, $f_n = 2f_{n-1} + n$ for $n \geq 2$. The following is a purported induction proof that $f_n = O(n)$.
>
> Proof. Base case: When $n = 1$, $f(n) = O(n)$ because $f(1) = O(1)$.
>
> Induction step: Assuming $f_{n-1} = O(n-1)$, we will prove $f_n = O(n)$:
>
> $$\begin{aligned} f_n &= 2f_{n-1} + n \\ &= 2 \cdot O(n-1) + n \quad \text{(induction hypothesis)} \\ &= O(3n-2) \\ &= O(n) \qquad\qquad \text{(dropping lower order term and constant coefficient)} \end{aligned}$$
>
> as desired.
>
> (b) Prove that $f(n) = O(2^n)$ by induction.

Official Answer

Source: physical PDF page 17 (2019-final.pdf).

We will show that $f(n) \leq 2 \cdot 2^n - n - 2$. Base case: When $n = 1$, $f(n) = 1 \leq 2 \cdot 2^1 - 1 - 2$. Induction step: Assuming $f_{n-1} \leq 2 \cdot 2^{n-1} - (n-1) - 2$, we will prove $f_n \leq 2 \cdot 2^n - n - 2$:

$$\begin{aligned} f_n &= 2f_{n-1} + n \\ &\leq 2(2 \cdot 2^{n-1} - (n-1) - 2) + n \quad \text{(induction hypothesis)} \\ &= 2 \cdot 2^n - n - 2. \end{aligned}$$

Comment: It is impossible to prove $f_n \leq c \cdot 2^n$ by induction, no matter how you choose the constant c. Interestingly, and somehow counter-intuitively, it is possible to prove the stronger claim $f(n) \leq c \cdot 2^n - n - 2$. Fundamentally, this is because a stronger claim will also make the induction hypothesis stronger, which sometimes makes the induction step easier to go through. This technique is known as "strengthening the induction hypothesis".

Supplementary Explanation

Answer. $f_n = 2^{n+1} - n - 2 \leq 2 \cdot 2^n$, so $f_n = O(2^n)$.

Derivation/proof. Prove the stronger exact identity by induction. At n=1, $2^2 - 1 - 2 = 1 = f_1$. Assume $f_{n-1} = 2^n - (n-1) - 2$. Then $f_n = 2[2^n - (n-1) - 2] + n = 2^{n+1} - n - 2$. Thus the identity holds for all $n \geq 1$, and immediately gives the fixed bound $f_n \leq 2 \cdot 2^n$. Equivalently, the source's strengthened inequality $f_n \leq 2 \cdot 2^n - n - 2$ propagates by the same algebra. Directly iterating the recurrence verifies its first values $1, 4, 11, 26$, matching the formula.

The first four values $1, 4, 11, 26$ satisfy both the recurrence and formula. Positivity together with $f_n \leq 2 \cdot 2^n$ gives fixed constant $C = 2$ and threshold $n_0 = 1$.

A068

Algorithms

2019 Final | Question 4(b) | Source page 6 | Difficulty 4 | 4 marks
Q-2019-FINAL-P04-b | Answer provenance: official

> (b) Give an example of two increasing functions $f(n)$ and $g(n)$ from the set of positive integers to the set of positive integers such that neither $f(n)$ is $O(g(n))$ nor $g(n)$ is $O(f(n))$. Justify your answer.

Official Answer

Source: physical PDF page 6 (2019-final.pdf).

We want the functions to play leap-frog, with first one much bigger than the other. Something like this will do the trick: Let $f(n) = n^{2\lfloor n/2 \rfloor + 1}$ and $g(n) = n^{2\lceil n/2 \rceil}$. Thus, $\{f(1) = 1^1, g(1) = 1^2\}, \{f(2) = 2^3, g(2) = 2^2\}, \{f(3) = 3^3, g(3) = 3^4\}, \{f(4) = 4^5, g(4) = 4^4\}$, and so on. Then for even n we have $f(n)/g(n) = n$, and for odd n we have $g(n)/f(n) = n$. Because both ratios are unbounded, neither function is big-O of the other.

Supplementary Explanation

Answer. $f(n) = n^{2\lfloor n/2 \rfloor + 1}$ and $g(n) = n^{2\lceil n/2 \rceil}$.

Derivation/proof. Both exponent sequences are positive and nondecreasing, and the base n strictly increases, so each function is strictly increasing on positive integers. For even n, the exponents are n+1 and n, so $f(n)/g(n) = n$. For odd n, the exponents are n and n+1, so $g(n)/f(n) = n$. Each ratio is therefore unbounded along an infinite subsequence. Any purported eventual constant big-O bound fails on its corresponding subsequence.

REFERENCE

Source index

Exercise	Knowledge-base question ID	Source page
A001	Q-2022-MID-P10-a-best	4
A002	Q-2022-MID-P10-a-worst	4
A003	Q-2022-MID-P10-b-best	4
A004	Q-2022-MID-P10-b-worst	4
A005	Q-2022-MID-P10-c-best	4
A006	Q-2021-MID-P08-d	3
A007	Q-2018-FINAL-P10-b	14
A008	Q-2017-FINAL-P03-b	5
A009	Q-2025F-FINAL-P08-i	13
A010	Q-2025F-FINAL-P08-ii	13
A011	Q-2025F-FINAL-P08-iii	13
A012	Q-2025F-FINAL-P08-v	13
A013	Q-2025S-FINAL-P05-a-i	6
A014	Q-2025S-FINAL-P05-b	6
A015	Q-2024F-FINAL-P09-a-i	23
A016	Q-2024F-FINAL-P09-a-ii	23
A017	Q-2024F-FINAL-P09-a-iii	23
A018	Q-2024F-FINAL-P09-a-iv	23
A019	Q-2024F-FINAL-P09-a-v	23
A020	Q-2024S-MID-P01	3
A021	Q-2023S-MID-P08-a	12
A022	Q-2023S-MID-P08-b	12
A023	Q-2023S-MID-P08-c	12
A024	Q-2023S-MID-P08-e	12

Exercise	Knowledge-base question ID	Source page
A025	Q-2021-MID-P08-b	3
A026	Q-2021-MID-P08-c	3
A027	Q-2021-MID-P09-A	4
A028	Q-2020-FINAL1-P06-a	5
A029	Q-2018-FINAL-P10-a	14
A030	Q-2018-FINAL-P10-c	14
A031	Q-2017-FINAL-P02	4
A032	Q-2017-FINAL-P03-a	5
A033	Q-2017-FINAL-P03-c	5
A034	Q-2015-FINAL-P09-recurrence	15
A035	Q-2015-FINAL-P09-theta	15
A036	Q-2014-FINAL-P09-theta	13
A037	Q-2013-FINAL-P08-theta	11
A038	Q-2025F-FINAL-P08-iv	13
A039	Q-2025F-FINAL-P09-a	14
A040	Q-2025F-FINAL-P09-b	14
A041	Q-2025S-FINAL-P05-a-ii	6
A042	Q-2025S-FINAL-P05-a-iii	6
A043	Q-2023F-FINAL-P10	16
A044	Q-2022-MID-P09	4
A045	Q-2022-MID-P10-c-worst	4
A046	Q-2021-FINAL1-P02	1
A047	Q-2021-MID-P08-a	3
A048	Q-2021-MID-P08-e	3
A049	Q-2021-MID-P09-B	4
A050	Q-2020-FINAL1-P05-b	3
A051	Q-2020-FINAL1-P06-b	5

Exercise	Knowledge-base question ID	Source page
A052	Q-2019-FINAL-P04-a	6
A053	Q-2018-FINAL-P10-d	14
A054	Q-2018-FINAL-P12-a	17
A055	Q-2018-FINAL-P12-b	17
A056	Q-2017-FINAL-P04-b-time	6
A057	Q-2017-FINAL-P04-c	6
A058	Q-2015-FINAL-P09-closed-form	15
A059	Q-2014-FINAL-P08	12
A060	Q-2014-FINAL-P09-closed-form	13
A061	Q-2013-FINAL-P08-closed-form	11
A062	Q-2013-FINAL-P09	12
A063	Q-2025S-FINAL-P05-c	6
A064	Q-2023F-FINAL-P08-a	12
A065	Q-2021-FINAL1-P03	2
A066	Q-2019-FINAL-BONUS-a	17
A067	Q-2019-FINAL-BONUS-b	17
A068	Q-2019-FINAL-P04-b	6